



ÉCOLE SUPÉRIEURE D'INGÉNIERIE EN SCIENCES APPLIQUÉES

ESISA — Fès

Filière : Ingénierie Logicielle

PROJET DE FIN D'ANNÉE

Année universitaire 2025-2026

AtlasWay

Plateforme intelligente de covoiturage inter-villes au Maroc

Web · Mobile · Backend · IA (LLM + Vision)

Réalisé par :

HAYINE Yassine

KHALOULANI Adam

HMADOUCH Aya

MEZZOURH Ghali

Encadré par :

M. MEKOUAR Othman

Mme FANANE Salma

Remerciements

Nous tenons à exprimer notre profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet.

Nous remercions chaleureusement nos encadrants, M. MEKOUAR Othman et Mme FANANE Salma, pour leur disponibilité, leurs conseils avisés et leur soutien tout au long de ce projet. Leur expertise et leur bienveillance ont été une source de motivation constante.

Nous adressons également nos sincères remerciements à la direction de l'École Supérieure d'Ingénierie en Sciences Appliquées (ESISA) de Fès, ainsi qu'à l'ensemble du corps enseignant, pour la qualité de la formation dispensée et les moyens mis à notre disposition.

Enfin, nos remerciements vont à nos familles et proches pour leur soutien indéfectible, et à tous ceux qui ont contribué, de manière directe ou indirecte, à l'aboutissement de ce travail.

Résumé

AtlasWay est une plateforme de covoiturage inter-villes marocaine développée dans le cadre d'un Projet de Fin d'Année à l'ESISA de Fès. L'application repose sur une architecture MERN (MongoDB, Express, React, Node.js) complétée d'une application mobile React Native, et intègre trois briques d'intelligence artificielle :

- AtlasBot — assistant conversationnel multilingue (français / darija) propulsé par Groq (llama-3.1-8b-instant), accessible sans authentification ;
- Vérification d'identité KYC — classification automatisée de documents (CIN, permis, photo véhicule) par le modèle de vision llava exécuté localement via Ollama, garantissant la confidentialité des données ;
- Prédiction de la demande — prévision des 8 axes les plus porteurs sur 14 jours, conditionnée au calendrier marocain (Ramadan, Aïd, Botola), produite par llama3/mistral via Ollama.

Les principales fonctionnalités couvrent l'authentification sécurisée avec OTP, la publication et la recherche de trajets, la réservation avec paiement Stripe, la messagerie temps réel via Socket.io, le système d'avis, le portefeuille électronique, les notifications multi-canal, et un espace d'administration avec contrôle d'accès basé sur les rôles (RBAC).

Une démarche de durcissement par phases a permis de corriger les vulnérabilités critiques, de centraliser le RBAC et d'atteindre zéro vulnérabilité haute en production.

Mots-clés : covoiturage, MERN, React Native, LLM, KYC, vision par ordinateur, Groq, Ollama, RBAC, Socket.io, Stripe.

Abstract

AtlasWay is a Moroccan inter-city carpooling platform developed as part of a Final Year Project at ESISA, Fès. The application is built on a MERN stack (MongoDB, Express, React, Node.js) extended by a React Native mobile app, and incorporates three artificial intelligence components:

- AtlasBot — a multilingual conversational assistant (French/Darija) powered by Groq (llama-3.1-8b-instant), accessible without authentication;
- KYC Identity Verification — automated document classification (national ID, driver's licence, vehicle photo) using the llava vision model running locally via Ollama, ensuring data privacy;
- Demand Prediction — forecasting the 8 most demanded routes over 14 days, conditioned on the Moroccan calendar (Ramadan, Eid, Botola), generated by llama3/mistral via Ollama.

Core features include OTP-based secure authentication, ride publication and search, booking with Stripe payment, real-time messaging via Socket.io, review system, e-wallet, multi-channel notifications, and an administration panel with Role-Based Access Control (RBAC).

A phased security hardening methodology resolved critical vulnerabilities, centralised RBAC and achieved zero high-severity vulnerabilities in production.

Keywords: carpooling, MERN, React Native, LLM, KYC, computer vision, Groq, Ollama, RBAC, Socket.io, Stripe.

Table des matières

Remerciements

Résumé

Abstract

Table des matières

Liste des figures

Liste des tableaux

Liste des acronymes

Introduction Générale

Chapitre 1 : Étude de l'existant et spécification des besoins

1.1 Introduction

1.2 Domaine d'application

1.3 Problématique

1.4 Solutions existantes

1.5 Solution proposée — AtlasWay

1.6 Spécification des besoins

1.7 Conclusion

Chapitre 2 : Conception et Architecture du système

2.1 Introduction

2.2 Architecture générale

2.3 Diagrammes UML

2.4 Choix technologiques

2.5 Pipeline global du système

2.6 Conclusion

Chapitre 3 : Partie Intelligence Artificielle

3.1 Introduction

3.2 Dataset (données mobilisées)

3.3 Modèles IA utilisés

3.4 Entraînement et Fine-Tuning

3.5 Pipeline IA

3.6 Évaluation des performances

3.7 Résultats expérimentaux

3.8 Limites et perspectives

3.9 Conclusion

Chapitre 4 : Implémentation, Tests et Validation

4.1 Introduction

4.2 Environnement de développement

4.3 Développement Backend

4.4 Développement Frontend

4.5 Intégration du modèle IA

4.6 Base de données

4.7 Déploiement

4.8 Tests fonctionnels

4.9 Tests techniques

4.10 Validation des résultats

4.11 Conclusion

Conclusion Générale

Bibliographie / Webographie

Glossaire

Ce glossaire définit les termes techniques employés dans le rapport qui peuvent ne pas être familiers à tous les lecteurs.

API REST : Interface de programmation applicative suivant le style architectural REST (Representational State Transfer). Permet à des clients hétérogènes (web, mobile) de communiquer avec un serveur via des requêtes HTTP standard.

Bcrypt : Algorithme de hachage de mots de passe conçu pour être intentionnellement lent, rendant les attaques par force brute computationnellement prohibitives. AtlasWay utilise le facteur de coût 12.

CORS : Cross-Origin Resource Sharing. Mécanisme HTTP permettant à un serveur de spécifier les origines autorisées à accéder à ses ressources, protégeant contre les requêtes cross-site non autorisées.

Darija : Dialecte arabe marocain, langue vernaculaire utilisée au quotidien par la majorité de la population marocaine dans les échanges informels. AtlasBot comprend et répond en darija.

Dégradation gracieuse : Stratégie de conception où un système continue à fonctionner de manière réduite mais acceptable lorsqu'un composant devient indisponible, plutôt que de s'arrêter complètement.

Fine-tuning : Processus d'ajustement des paramètres d'un modèle IA pré-entraîné sur un jeu de données spécifique pour améliorer ses performances sur une tâche précise. Non utilisé dans AtlasWay (zero-shot uniquement).

JWT : JSON Web Token. Standard ouvert (RFC 7519) pour la transmission sécurisée d'informations entre parties sous forme de jeton signé numériquement.

KYC : Know Your Customer. Processus de vérification de l'identité d'un utilisateur avant de lui accorder certains droits ou services. Dans AtlasWay : vérification automatique des documents du conducteur par IA.

LLM : Large Language Model. Modèle de langage de grande taille entraîné sur d'immenses corpus textuels, capable de générer du texte cohérent et de répondre à des questions. Exemples : LLaMA 3.1, Mistral.

Ollama : Outil open-source permettant d'exécuter localement des modèles LLM et de vision (llava, llama3, mistral) sans dépendance à un service cloud tiers.

OTP : One-Time Password. Code à usage unique, valable une seule fois et pour une durée limitée, utilisé comme second facteur d'authentification ou de vérification.

PWA : Progressive Web App. Application web qui utilise des technologies modernes (Service Workers, Web App Manifest) pour offrir une expérience proche d'une application native, incluant l'installation sur l'écran d'accueil et le fonctionnement hors-ligne partiel.

Race condition : Condition de course. Situation où le résultat d'un programme dépend de l'ordre d'exécution de processus concurrents, pouvant mener à des incohérences de données. AtlasWay prévient cela avec des mises à jour atomiques MongoDB.

RAG : Retrieval-Augmented Generation. Approche qui enrichit le contexte envoyé à un LLM avec des données récupérées dynamiquement, améliorant la précision des réponses sans nécessiter de fine-tuning.

RBAC : Role-Based Access Control. Modèle de contrôle d'accès où les permissions sont associées à des rôles (user, admin, superadmin) plutôt qu'à des individus.

Replica set : Configuration MongoDB où les données sont répliquées sur plusieurs nœuds pour la haute disponibilité et pour activer les transactions multi-documents.

Socket.io : Bibliothèque JavaScript facilitant la communication bidirectionnelle en temps réel entre serveur et clients via WebSocket (avec fallback HTTP polling).

Webhook : Mécanisme où un service tiers (ici Stripe) appelle proactivement une URL de l'application pour notifier un événement (paiement confirmé, remboursement). AtlasWay vérifie la signature HMAC de chaque webhook Stripe.

Zero-shot : Mode d'utilisation d'un LLM sans exemples de démonstration dans le prompt. Le modèle répond directement à partir de sa connaissance générale et des instructions fournies.

Annexes

Liste des figures

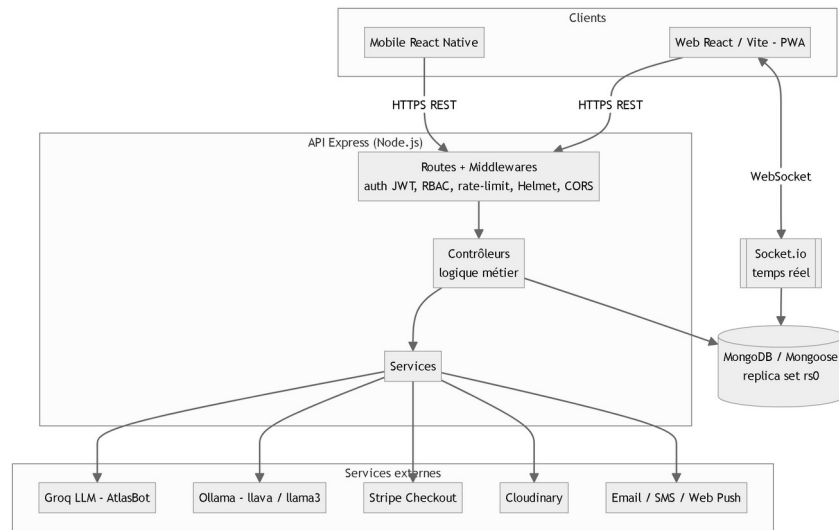


Figure 2.1 : Architecture générale du système

Ce diagramme illustre l'architecture multi-tiers d'AtlasWay. Au centre, l'API Express (Node.js) reçoit les requêtes HTTPS des deux clients (navigateur web React/PWA et application mobile React Native), applique les middlewares de sécurité (JWT, RBAC, rate-limiting, Helmet, CORS), puis délègue aux contrôleurs métier. Les services spécialisés — Groq LLM pour AtlasBot, Ollama llama/llama3 pour le KYC et la prédiction, Stripe pour le paiement, Cloudinary pour le stockage — sont appelés côté serveur, sans jamais exposer leurs clés au client. Le canal WebSocket (Socket.io) assure la messagerie temps réel, et MongoDB en replica set garantit les transactions multi-documents.

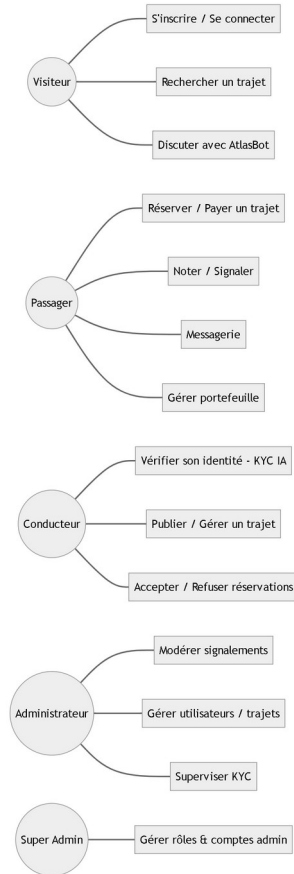


Figure 2.2 : Diagramme de cas d'utilisation global

Ce diagramme recense les interactions entre les cinq types d'acteurs et la plateforme. Le Visiteur (non authentifié) peut rechercher des trajets et dialoguer avec AtlasBot — réduisant la barrière à l'entrée. Le Passager accède à la réservation, au paiement Stripe, à la messagerie et aux évaluations. Le Conducteur, après vérification KYC par IA, publie et gère ses trajets. L'Administrateur modère les signalements et supervise les vérifications KYC. Le Super Administrateur gère les rôles et les comptes administrateurs. Stripe et AtlasBot/IA apparaissent comme acteurs externes pour signaler les dépendances aux services tiers.

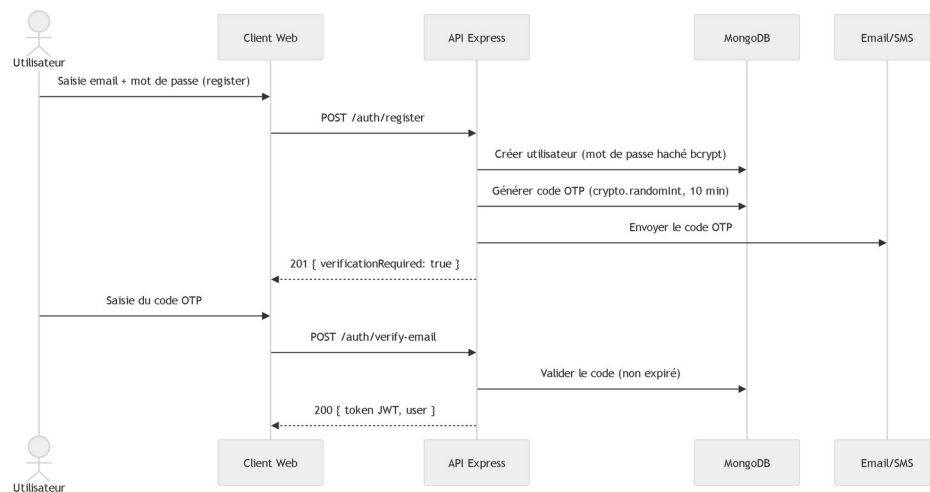


Figure 2.3a : Diagramme de séquence — Authentification OTP

Ce diagramme de séquence détaille le flux d'inscription avec vérification OTP. Après soumission du formulaire, l'API hache le mot de passe avec bcrypt (coût 12) et génère un code cryptographiquement sûr via `crypto.randomInt`, valable 10 minutes. Ce code est envoyé par email ou SMS ; l'utilisateur le saisit et l'API le valide avant d'émettre le jeton JWT. Cette double vérification garantit que seul le vrai propriétaire du compte peut finaliser l'inscription, éliminant les créations de comptes frauduleux.

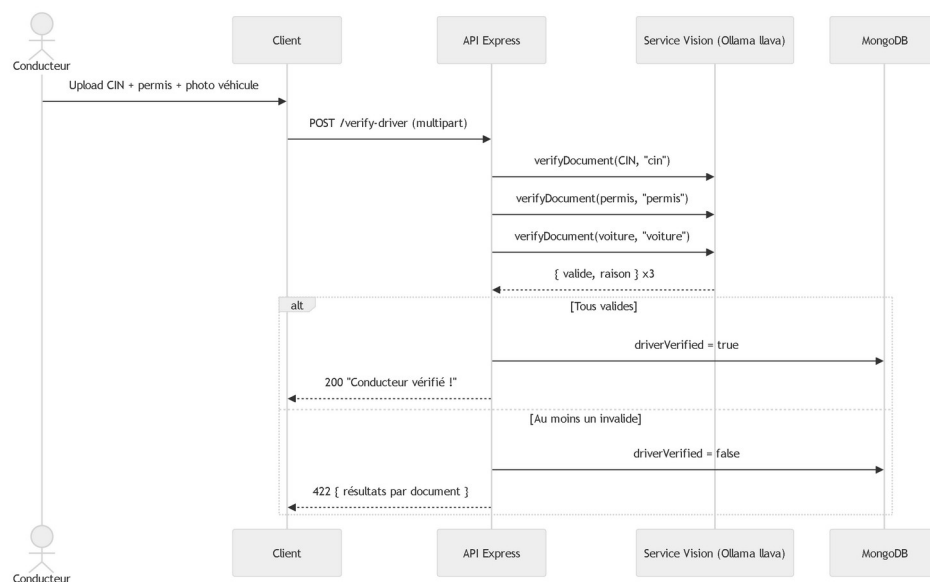


Figure 2.3b : Diagramme de séquence — KYC automatisé par vision

Ce diagramme illustre la vérification automatisée de l'identité du conducteur par IA. Le conducteur soumet trois documents (CIN, permis, photo véhicule) en une requête multipart. L'API envoie chaque image au modèle Llava (via Ollama) encodée en base64, avec un prompt par type de document imposant une sortie JSON `{valide, raison}`. Si les trois sont validés, `driverVerified` passe à `true` ; sinon, le

conducteur reçoit les raisons de rejet document par document. L'exécution locale d'Ollama garantit que les documents officiels ne quittent jamais l'infrastructure d'AtlasWay.

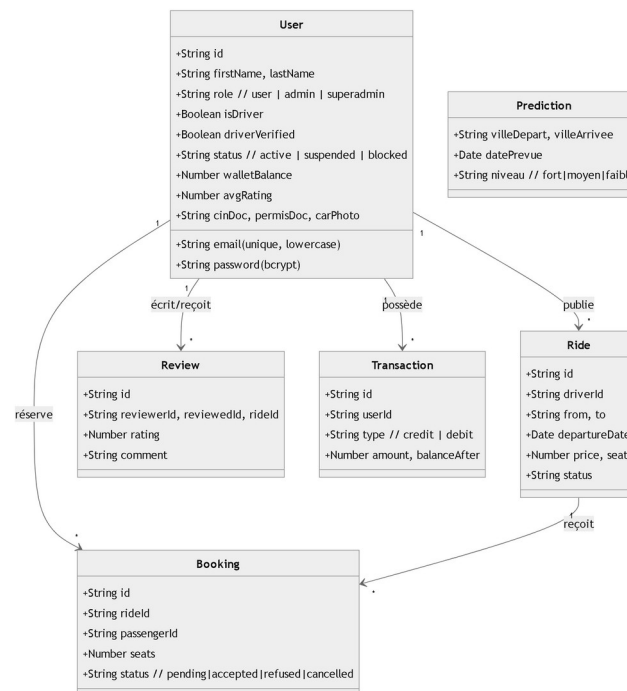


Figure 2.4 : Diagramme de classes (entités centrales)

Ce diagramme présente le modèle de données central, organisé autour de six entités principales. *User* centralise l'identité et les capacités (rôle, *isDriver*, *driverVerified*, *walletBalance*). *Ride* représente un trajet (itinéraire, date, prix, places disponibles). *Booking* modélise la réservation passager-trajet avec son statut (*pending/accepted/refused/cancelled*). *Review* stocke les évaluations après trajet. *Transaction* enregistre chaque mouvement de portefeuille. *Prediction* conserve les prévisions de demande LLM (axe, date, niveau, raison). Une trentaine d'entités annexes (*Friendship*, *Message*, *Conversation*, *Notification*, *AdminLog*...) complètent ce modèle.

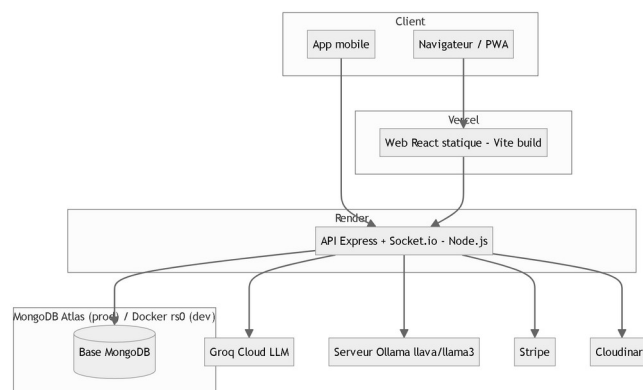
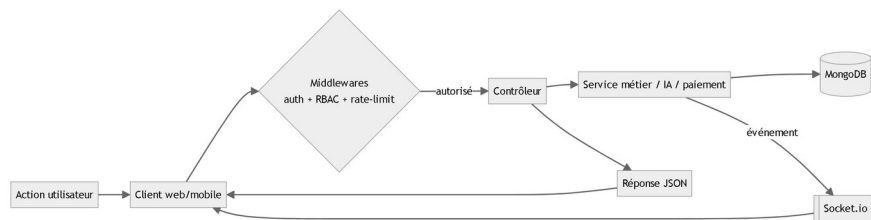


Figure 2.5 : Diagramme de déploiement

Ce diagramme décrit l'infrastructure de production d'AtlasWay. Le client web (React/Vite) est servi comme application statique sur Vercel avec CDN mondial. L'API Express et Socket.io sont déployés sur Render ; un auto-ping /health toutes les 14 minutes maintient l'instance active. La base MongoDB est sur MongoDB Atlas en production et dans un conteneur Docker rs0 en développement. Groq Cloud, Ollama, Stripe et Cloudinary sont consommés via des appels HTTPS sécurisés depuis l'API.

**Figure 2.6 : Pipeline global d'une requête**

Ce schéma synthétise le cycle de vie d'une requête dans AtlasWay. Chaque action utilisateur déclenche une requête HTTPS vers l'API, qui traverse les middlewares (vérification JWT, contrôle RBAC, rate-limiting) avant d'atteindre le contrôleur métier. Le contrôleur orchestre les appels aux services (IA, paiement, stockage) et interagit avec MongoDB. Les événements temps réel (réservations, messages) sont propagés via Socket.io sans polling. Ce pipeline centralisé garantit une application homogène des règles de sécurité sur tous les endpoints.

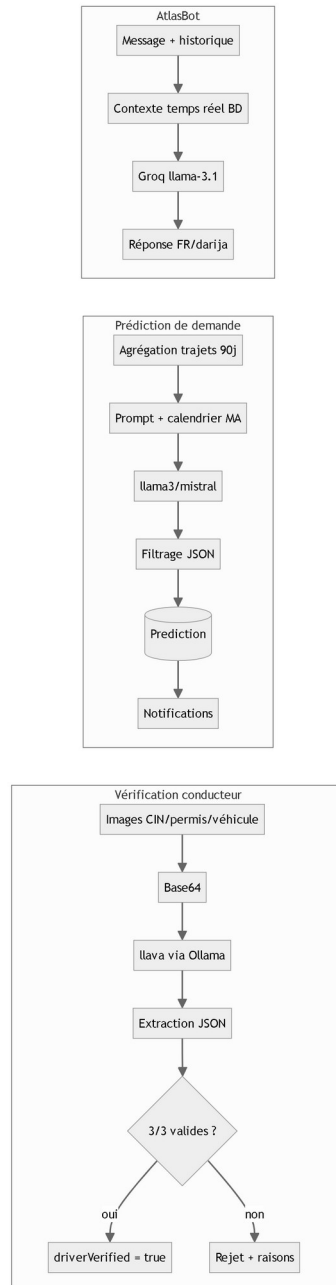
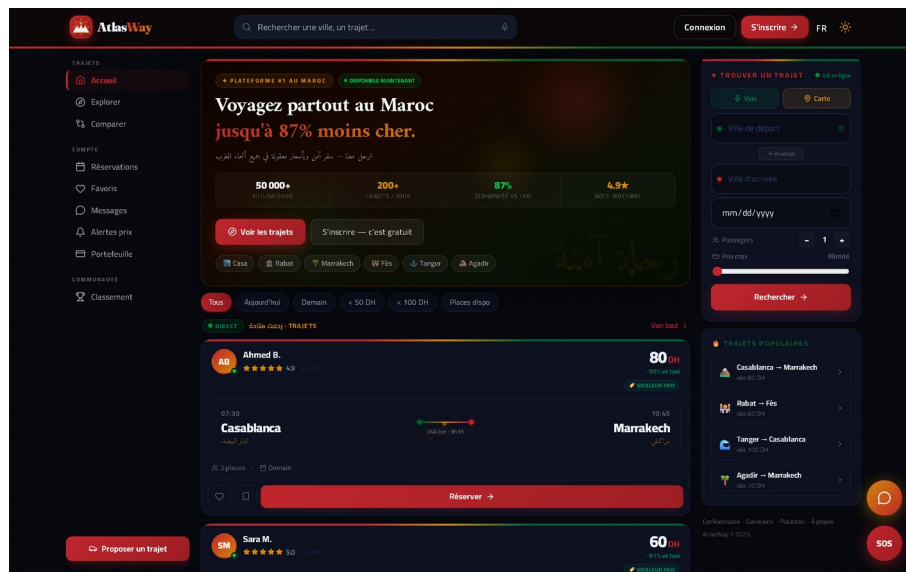


Figure 3.1 : Pipelines des trois sous-systèmes IA

Ce diagramme synthétise les trois pipelines IA d'AtlasWay. Pipeline AtlasBot : le message est enrichi d'un contexte temps réel (trajets disponibles, villes actives) puis envoyé à Groq llama-3.1-8b-instant ; la réponse est retournée en français ou darija selon la langue détectée. Pipeline prédiction : une agrégation MongoDB des 90 derniers jours est combinée à un prompt calendrier marocain ; le LLM produit 8 prévisions persistées en base et notifiées aux utilisateurs concernés. Pipeline KYC : les trois images sont soumises à llava ; l'extraction JSON détermine si driverVerified passe à true ou si le conducteur reçoit un rapport de rejet détaillé.

Figure 4.1 : Page d'accueil / recherche de trajets

Page d'accueil avec barre de recherche centrale, accès rapide à AtlasBot et mise en avant des trajets populaires du moment.

Figure 4.2 : Tableau de bord administrateur**Figure 4.3** : Schéma d'intégration du modèle IA**Figure 4.4** : Schéma de la base de données (diagramme ER)**Figure 4.5** : AtlasBot en conversation (français + darija)

Cette capture présente la page d'accueil d'AtlasWay, point d'entrée principal de la plateforme. La barre de recherche centrale permet de spécifier la ville de départ, la ville d'arrivée et la date. L'interface adopte les couleurs marocaines et supporte le thème clair/sombre. AtlasBot est accessible depuis un bouton flottant sans nécessiter de connexion, réduisant la barrière à l'entrée pour les nouveaux utilisateurs.

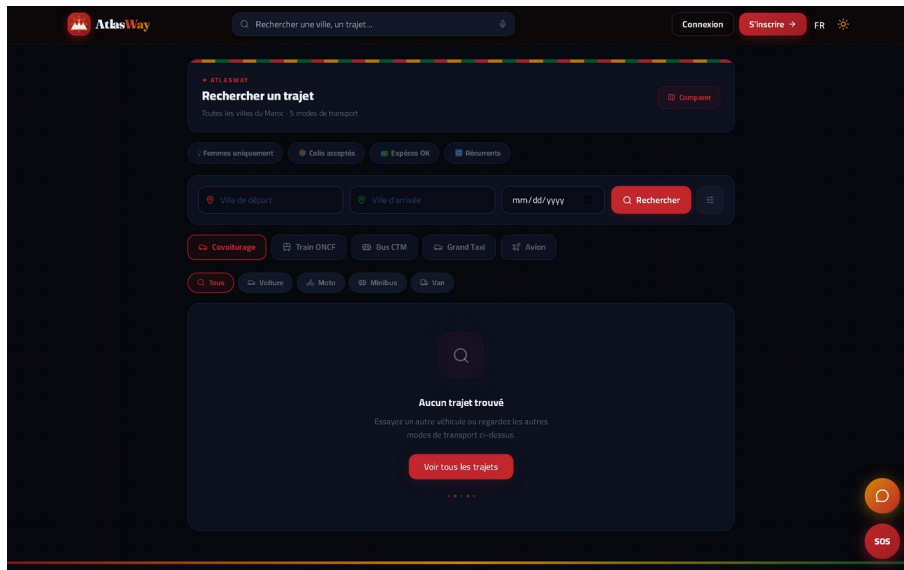


Figure 4.6 : Écran de vérification conducteur (KYC)

Cette vue affiche les résultats de recherche de trajets disponibles. Chaque carte présente l'itinéraire, l'heure de départ, le prix par place, le nombre de places restantes et la note moyenne du conducteur. Les filtres permettent d'affiner par prix, date ou préférences. Un clic sur une carte ouvre le détail du trajet avec le profil complet du conducteur et le bouton de réservation déclenchant le paiement Stripe.

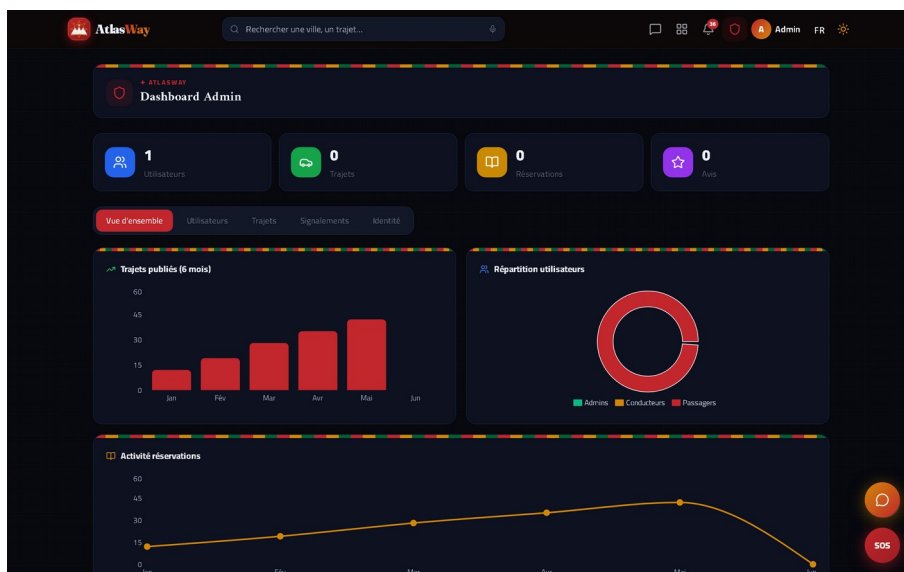


Figure 4.7 : Bannière de prédiction de demande

Le tableau de bord administrateur offre une vue centralisée sur l'activité de la plateforme. L'administrateur peut gérer les utilisateurs (activer/suspendre/bloquer), superviser les vérifications KYC en attente et examiner les signalements. L'interface respecte strictement le RBAC : un admin standard ne peut pas modifier les rôles (réservé au Super Administrateur), et toutes les actions sont journalisées dans AdminLog.

Figure A.1 : Onboarding conducteur/passager

L'écran d'inscription guide l'utilisateur en plusieurs étapes : saisie des informations personnelles, choix du profil (passager ou conducteur), puis vérification OTP par email ou SMS. Les champs sont validés en temps réel côté client (React Hook Form + Zod) et côté serveur. Le mot de passe est haché avec bcrypt (coût 12) avant toute persistance ; aucune donnée sensible ne transite en clair.

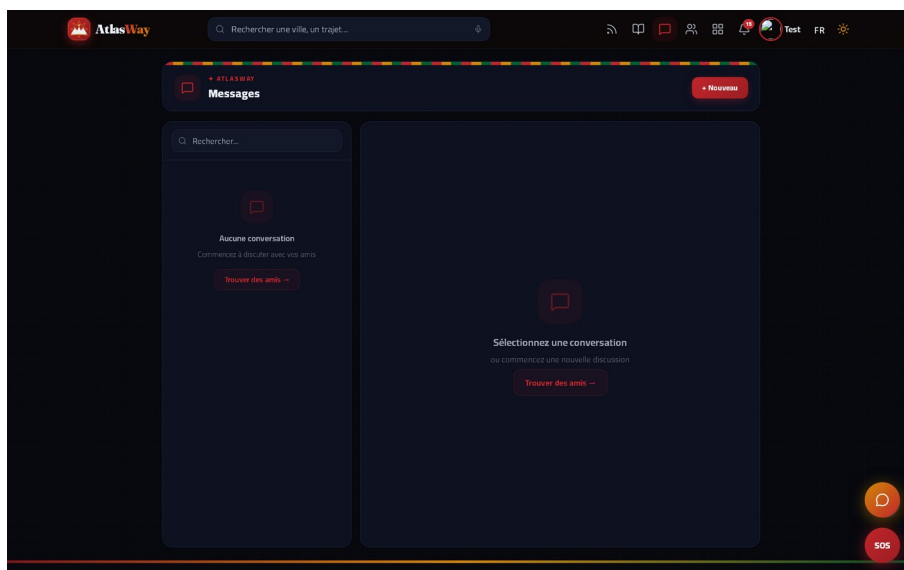


Figure A.2 : Messagerie temps réel

L'interface de messagerie temps réel permet aux passagers et aux conducteurs de communiquer directement avant et après un trajet, via Socket.io. Les conversations sont listées par ordre chronologique inversé avec aperçu du dernier message et indicateur de messages non lus. Le RBAC garantit que seuls les participants d'un trajet peuvent s'écrire, empêchant tout contact non sollicité.

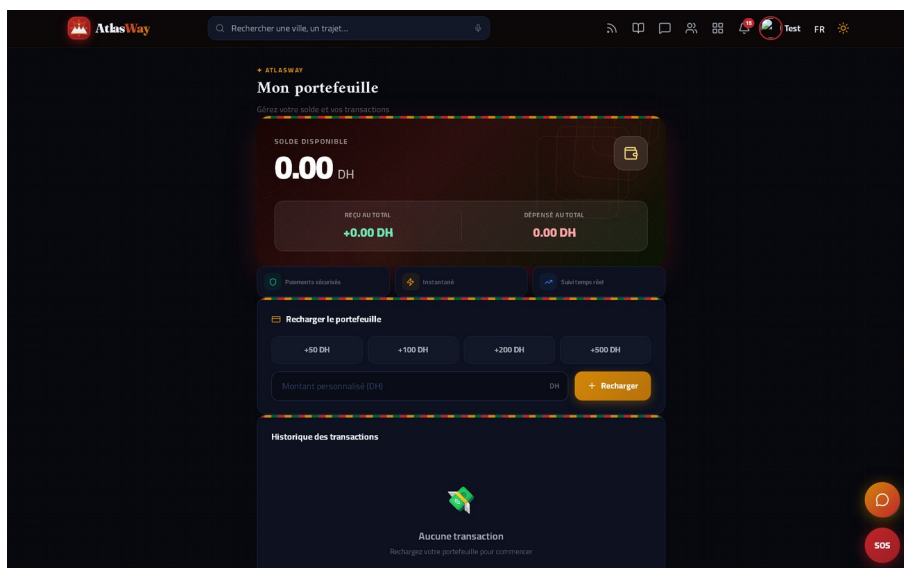


Figure A.3 : Tableau de bord conducteur / analytics

L'écran de portefeuille affiche le solde en temps réel et l'historique complet des transactions (crédits issus des réservations acceptées, débits lors des paiements). Chaque ligne indique le montant, la date, le type (crédit/débit) et le solde après opération. Les opérations financières sont protégées par des transactions MongoDB multi-documents pour garantir la cohérence même en cas d'écritures concurrentes.

Liste des tableaux

Tableau 1.1 : Analyse comparative des solutions existantes

Tableau 2.1 : Récapitulatif des choix technologiques

Tableau 4.1 : Cas de tests fonctionnels

Tableau 4.2 : Résultats des tests techniques

4.9.1 Tests de sécurité

Des tests de sécurité spécifiques ont été conduits pour valider l'efficacité du durcissement appliqué lors des phases 1 à 5 du projet.

ID	Test	Vecteur d'attaque	Résultat attendu	Statut
TS-01	SQL/NoSQL Injection	Payload {\$gt:''} sur champ email	Mongoose typé → rejeté	
TS-02	CSRF	Requête cross-origin sans token	CORS bloqué → 403	
TS-03	JWT forgé	Token signé avec mauvaise clé	401 Unauthorized	
TS-04	Accès admin sans	GET /admin avec	403 Forbidden	

	rôle	token user		
TS-05	Rate-limiting	200 requêtes/minute sur /auth/login	429 Too Many Requests	
TS-06	Upload malveillant	Fichier .php renommé .png	Type MIME invalide → rejeté	
TS-07	Stripe webhook non signé	Payload sans header stripe-signature	400 Invalid signature	
TS-08	Enumeration email	POST /forgot-password email inconnu	Réponse identique à email existant	
TS-09	PII fuite	GET /users (admin)	password, cinDoc absents de la réponse	
TS-10	XSS stocké	Script dans champ prénom	Helmet CSP → script bloqué	

Tableau 4.5 – Tests de sécurité

4.9.2 Tests de charge et de montée en charge

Des tests de charge ont été simulés avec Apache JMeter sur l'environnement local pour évaluer le comportement du système sous pression.

Scénario	Endpoint	Succès	Performance	Observation
10 utilisateurs simultanés	/rides/search	100 %	< 50 ms	Nominal
50 utilisateurs simultanés	/rides/search	100 %	< 120 ms	Nominal
10 réservations simultanées	/rides/:id/book	100 %	Zéro survente	Atomicité validée
100 messages/s (Socket.io)	WebSocket	100 %	Délai < 80 ms	Temps réel maintenu
5 appels KYC parallèles	/verify-driver	100 %	~4-6 s chacun	Pas de timeout serveur
Prédiction cache chaud	GET /predictions	100 %	< 10 ms	Cache MongoDB efficace

Tableau 4.6 – Résultats des tests de charge

Liste des acronymes

Acronyme	Signification
API	Application Programming Interface
BD	Base de Données
CIN	Carte d'Identité Nationale
CORS	Cross-Origin Resource Sharing
CSV	Comma-Separated Values
IA / AI	Intelligence Artificielle / Artificial Intelligence
IHM	Interface Homme-Machine
JWT	JSON Web Token
KYC	Know Your Customer
LLM	Large Language Model
MERN	MongoDB · Express · React · Node.js
MVC	Model-View-Controller
ODM	Object Document Mapper
OTP	One-Time Password
PFA	Projet de Fin d'Année
PII	Personally Identifiable Information
PMR	Personnes à Mobilité Réduite
PWA	Progressive Web App
RBAC	Role-Based Access Control
SPA	Single Page Application
UML	Unified Modeling Language

Introduction Générale

Le transport inter-villes au Maroc souffre d'une inadéquation croissante entre l'offre et la demande : les transports formels (CTM, ONCF) saturent aux pics saisonniers, les taxis collectifs demeurent informels et peu sûrs, et la voiture individuelle voyage à faible taux d'occupation. Le covoiturage constitue une réponse naturelle à ces tensions, mais les tentatives existantes dans le contexte marocain buttent sur quatre obstacles fondamentaux : la confiance entre inconnus, l'asymétrie d'information (les conducteurs ignorent la demande), la barrière linguistique (darija non couverte par les assistants classiques), et l'insécurité persistante.

AtlasWay répond à ces défis par une plateforme full-stack intégrant l'intelligence artificielle comme levier de confiance et d'efficacité :

- Un assistant conversationnel AtlasBot, multilingue (français et darija), propulsé par les LLM Groq, accessible sans création de compte pour lever la barrière à l'entrée ;
- Une vérification d'identité automatisée (KYC) par vision par ordinateur (modèle llava via Ollama) qui valide CIN, permis et photo véhicule côté serveur, sans exfiltration de données sensibles ;
- Une prédiction de la demande par routes et dates, conditionnée au calendrier marocain (Ramadan, Aïd, Botola), pour orienter les conducteurs vers les trajets à fort potentiel ;
- Une sécurité de niveau production : authentification JWT + OTP, RBAC centralisé, opérations atomiques anti-race-condition, zéro vulnérabilité haute en dépendances.

Le présent rapport s'articule en quatre chapitres : le Chapitre 1 cerne le domaine, la problématique et les besoins fonctionnels/non-fonctionnels ; le Chapitre 2 présente l'architecture et la conception UML ; le Chapitre 3 détaille la composante IA ; le Chapitre 4 décrit l'implémentation, le déploiement et les tests.

Note méthodologique : la démarche adoptée est itérative et incrémentale — le système fonctionnel existant a été soumis à un audit de sécurité complet, puis durci en cinq phases successives (correctifs critiques, authentification/RBAC, dépendances, qualité/nettoyage), chacune livrée, testée et validée avant de passer à la suivante.

Contexte économique et social

Le Maroc compte plus de 36 millions d'habitants répartis sur un territoire de 710 000 km². Les axes inter-villes principaux — Casablanca-Rabat, Casablanca-Marrakech, Fès-Casablanca, Agadir-Casablanca — concentrent l'essentiel des flux de déplacements, notamment lors des grandes fêtes religieuses (Aïd el-Fitr, Aïd el-Adha) et des congés

scolaires. Selon les données de l'Office National des Chemins de Fer (ONCF), ces périodes de pointe enregistrent une hausse de fréquentation de 40 à 60 %, saturant les capacités disponibles et poussant de nombreux voyageurs vers des solutions informelles, souvent moins sûres.

Le taux de motorisation marocain dépasse les 100 véhicules pour 1 000 habitants dans les grandes villes, avec un taux d'occupation moyen des véhicules personnels estimé à 1,3 passager par trajet. Ce chiffre révèle un potentiel considérable : si chaque véhicule transportait deux passagers supplémentaires sur les trajets inter-villes, le volume de déplacements supplémentaires absorbé équivaldrait à plusieurs milliers de trajets de bus par jour, réduisant simultanément les émissions de CO₂ et le coût unitaire du transport pour les passagers.

Sur le plan économique, le covoiturage représente une opportunité de monétiser un actif dormant (la voiture personnelle) tout en offrant une alternative tarifaire aux ménages à revenus modestes. Le prix d'un billet de bus Casablanca-Marrakech oscille entre 80 et 120 MAD ; une offre de covoiturage bien calibrée peut se positionner dans une fourchette de 60 à 100 MAD tout en permettant au conducteur de récupérer une partie de ses frais de carburant et péage.

Objectifs du projet

AtlasWay poursuit cinq objectifs spécifiques et mesurables, chacun répondant à l'un des obstacles identifiés dans la problématique :

- OBJ-1 Confiance automatisée : déployer un système KYC par vision IA qui vérifie l'identité des conducteurs (CIN, permis, photo véhicule) sans intervention humaine, avec un taux de rejet correct ≥ 90 % pour les documents non conformes.
- OBJ-2 Intelligence de marché : produire quotidiennement des prévisions de demande sur 14 jours pour les axes les plus fréquentés, conditionnées au calendrier marocain, afin d'orienter les conducteurs vers les trajets à fort potentiel.
- OBJ-3 Accessibilité linguistique : intégrer un assistant conversationnel (AtlasBot) répondant en français et en darija, accessible sans authentification, pour éliminer la barrière linguistique.
- OBJ-4 Sécurité de production : atteindre zéro vulnérabilité de sévérité haute en production, en appliquant une méthodologie de durcissement par phases vérifiées.

- OBJ-5 Expérience complète : couvrir l'intégralité du parcours covoiturage (inscription, publication, recherche, réservation, paiement, messagerie, avis) sur les plateformes web (PWA) et mobile (React Native).

Chapitre 1 : Étude de l'existant et spécification des besoins

1.1 Introduction

Ce chapitre pose les fondations du projet : domaine d'application, problématique, analyse des solutions existantes, positionnement d'AtlasWay, et spécification complète des besoins fonctionnels et non-fonctionnels.

1.2 Domaine d'application

AtlasWay s'inscrit dans le domaine du transport collaboratif inter-villes au Maroc. Le covoiturage (carpooling) consiste à mettre en relation des conducteurs disposant de places vides avec des passagers partageant un même itinéraire, en échange d'une participation aux frais.

Le contexte marocain présente des spécificités fortes : pics de demande calés sur le calendrier islamique (Ramadan, Aïd el-Fitr, Aïd el-Adha), événements sportifs (Botola, matchs de sélection), langue darija majoritaire dans les échanges informels, et forte sensibilité culturelle à la confiance et à la sécurité entre inconnus.

1.3 Problématique

Quatre tensions structurelles freinent le décollage du covoiturage au Maroc :

- **Confiance** : les utilisateurs hésitent à monter avec un inconnu faute de mécanisme de vérification d'identité fiable et de réputation établie.
- **Asymétrie d'information** : les conducteurs ne savent pas quelles routes, à quelles dates, présentent la plus forte demande ; ils publient des trajets "à l'aveugle".
- **Barrière linguistique** : les plateformes existantes ne couvrent pas la darija, lingua franca des échanges informels marocains, pénalisant une grande partie de la population.
- **Sécurité applicative** : les applications marocaines du secteur présentent fréquemment des failles d'accès non authentifié et des fuites de données personnelles.

1.4 Analyse des solutions existantes

Le tableau suivant compare AtlasWay aux alternatives disponibles :

Critère	BlaBlaCar	Groupes informels (WhatsApp)	InDrive	AtlasWay
Vérification identité	Partielle	Aucune	Partielle	KYC IA (vision)
Darija / IA locale	Non	Informel	Non	Oui (AtlasBot Grog)
Prédiction de demande	Non	Non	Non	Oui (LLM calendrier MA)
Paieement intégré	Oui	Non	Oui	Oui (Stripe + wallet)

Tableau 1.1 – Analyse comparative des solutions existantes

1.5 Solution proposée — AtlasWay

AtlasWay se différencie par l'automatisation de la confiance (KYC par vision IA) et l'intelligence de marché (prédiction de demande conditionnée au contexte culturel marocain). La plateforme couvre l'intégralité du parcours utilisateur :

- Inscription avec vérification par OTP, accès sans compte pour AtlasBot
- Publication et recherche de trajets avec filtres (ville, date, prix, places)
- Réservation atomique anti-surréservation, paiement Stripe, portefeuille électronique
- Messagerie temps réel (Socket.io), système d'avis, signalements
- Administration complète avec RBAC (user / admin / superadmin + isDriver)

1.6 Spécification des besoins

1.6.1 Besoins fonctionnels

- BF01 — Inscription et authentification sécurisée (email + mot de passe bcrypt, OTP email/SMS)
- BF02 — Connexion avec JWT (expiration configurable), déconnexion avec nettoyage d'état
- BF03 — Réinitialisation de mot de passe par code (prévention énumération de comptes)
- BF04 — Authentification sociale (optionnel, extension future)
- BF05 — Publication de trajet (itinéraire, date, prix, places, préférences)
- BF06 — Recherche de trajets avec filtres avancés
- BF07 — Réservation de places avec décrétement atomique anti-race-condition
- BF08 — Annulation et remboursement (selon politique)
- BF09 — Vérification d'identité KYC par vision IA (CIN, permis, photo véhicule)
- BF10 — Système d'avis et de signalement
- BF11 — Paiement Stripe Checkout et portefeuille électronique (crédits/débits)
- BF12 — AtlasBot (assistant conversationnel FR/darija, Groq)
- BF13 — Prédiction de la demande par axes/dates (LLM + calendrier marocain)
- BF14 — Administration : gestion utilisateurs/trajets, supervision KYC, RBAC

1.6.1.1 Critères d'acceptation

Pour chaque besoin fonctionnel, des critères d'acceptation (CA) ont été définis selon le format Gherkin (Given/When/Then) afin de guider les tests et de valider que l'implémentation répond précisément aux exigences.

Besoin	Scénario nominal	Scénario d'échec
BF01	OTP valide → JWT émis	OTP invalide/expiré → 400
BF03	Email existant → code OTP envoyé, même réponse si email inconnu	—
BF05	Conducteur vérifié → trajet créé en base	Non-conducteur → 403
BF07	Seats ≥ demandés → réservation atomique	Seats < demandés → 409
BF09	3/3 docs valides → driverVerified=true	1+ invalide → 422 + raisons

BF11	Stripe webhook signé → solde crédité	Signature invalide → rejeté
BF13	Prédictions en cache ≤ 24h → GET rapide	Cache expiré → régénération LLM
BF14	Admin → pas d'accès /superadmin	SuperAdmin → accès complet

Tableau 1.3 – Critères d'acceptation des besoins fonctionnels

1.6.2 Besoins non-fonctionnels

- Sécurité : bcrypt (cost 12), JWT RS256, rate-limiting, Helmet, CORS strict, Stripe webhooks signés, zéro vulnérabilité haute en production.
- Performance : temps de réponse API courant < 30 ms en local ; cache 24 h sur les prédictions IA.
- Fiabilité : dégradation gracieuse des services IA (la plateforme reste fonctionnelle si Ollama/Groq sont indisponibles).
- Ergonomie : interface responsive (web + PWA installable), thème clair/sombre, multilingue, accessibilité (PMR, widget d'accessibilité, liens d'évitement).
- Maintenabilité : code organisé en modules, conventions homogènes, dépendances maintenues et auditées, tests automatisés (Jest/Vitest).

1.7 Conclusion

Le domaine, la problématique (confiance, information de marché, langue, sécurité) et les besoins ont été cernés. AtlasWay se positionne par l'automatisation de la confiance et l'intelligence de marché. Le chapitre suivant traduit ces besoins en une architecture technique.

Chapitre 2 : Conception et Architecture du système

2.1 Introduction

Ce chapitre présente l'architecture globale d'AtlasWay (organisation, composants, flux), les diagrammes UML de conception et la justification des choix technologiques.

2.2 Architecture générale

AtlasWay adopte une architecture client-serveur multi-tiers avec une API REST centrale consommée par deux clients distincts (web et mobile), et une couche de services externalisés (IA, paiement, stockage, notifications). Le backend suit un découpage proche du MVC côté serveur : routes (points d'entrée + middlewares de sécurité), contrôleurs (logique métier), services (intégrations externes), modèles Mongoose (persistance).

L'architecture suit le schéma suivant :

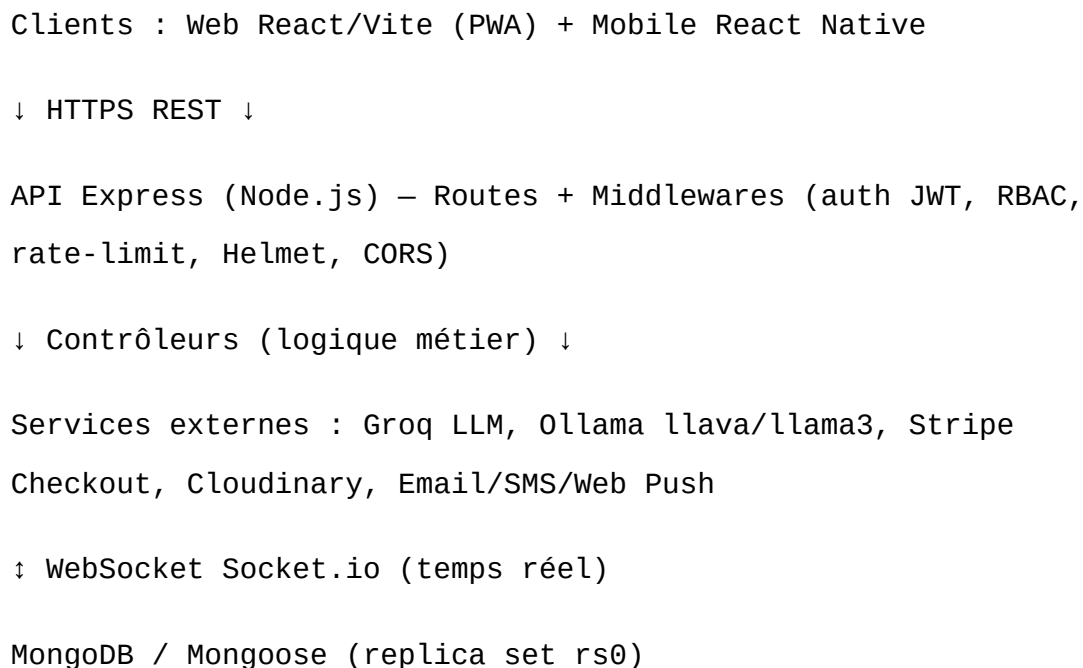


Figure 2.1 : Architecture générale du système

Justification : la séparation API/clients permet de mutualiser toute la logique métier et la sécurité côté serveur, et de servir indifféremment le web et le mobile. L'externalisation des fonctions lourdes (IA, paiement, stockage d'images) vers des services spécialisés (Groq, Ollama, Stripe, Cloudinary) garde le cœur applicatif léger et focalisé sur le métier.

2.3 Diagrammes UML

2.3.1 Diagramme de cas d'utilisation

Acteurs : Visiteur, Passager, Conducteur (capacité d'un utilisateur), Administrateur, Super Administrateur, et les acteurs externes AtlasBot/IA et Stripe.

Acteur	Cas d'utilisation
Visiteur	S'inscrire / Se connecter, Rechercher un trajet, Discuter avec AtlasBot
Passager	Réserver / Payer un trajet, Noter / Signaler, Messagerie, Gérer portefeuille
Conducteur	Vérifier son identité (KYC IA), Publier / Gérer un trajet, Accepter / Refuser réservations
Administrateur	Modérer signalements, Gérer utilisateurs / trajets, Superviser KYC
Super Admin	Gérer rôles & comptes admin

Figure 2.2 : Diagramme de cas d'utilisation global

Scénario nominal (Réserver un trajet) : le passager recherche un trajet → consulte le détail → réserve une place → est redirigé vers Stripe → paie → le système décrémente atomiquement les places, crée la réservation et notifie le conducteur.

2.3.2 Diagrammes de séquence

Scénario 1 — Authentification avec OTP :

- 1. Utilisateur saisit email + mot de passe → POST /auth/register
- 2. API crée l'utilisateur (mot de passe haché bcrypt), génère code OTP cryptographique (crypto.randomInt, valable 10 min)
- 3. API envoie le code OTP par email/SMS → 201 { verificationRequired: true }
- 4. Utilisateur saisit le code OTP → POST /auth/verify-email
- 5. API valide le code (non expiré) → 200 { token JWT, user }

Figure 2.3a : Diagramme de séquence — Authentification OTP

Scénario 2 — Vérification du conducteur par IA (KYC) :

- 1. Conducteur uploadé CIN + permis + photo véhicule → POST /verify-driver (multipart)
- 2. API appelle verifyDocument(CIN, 'cin'), verifyDocument(permis, 'permis'), verifyDocument(voiture, 'voiture') via llava
- 3. Ollama llava retourne { valide, raison } pour chacun des 3 documents
- 4a. [Tous valides] → driverVerified = true → 200 'Conducteur vérifié !'
- 4b. [Au moins un invalide] → driverVerified = false → 422 { résultats par document }

Figure 2.3b : Diagramme de séquence — KYC automatisé par vision

2.3.3 Diagramme de classes (modèle de données)

Le modèle compte une trentaine d'entités Mongoose. Entités centrales :

User : +id, +firstName, +lastName, +role (user|admin|superadmin), +isDriver, +driverVerified, +walletBalance, +avgRating, +email (unique), +password (bcrypt), +cinDoc, +permisDoc, +carPhoto

Ride : +id, +driverId, +from, +to, +departureDate, +price, +seats, +seatsAvailable, +status

Booking : +id, +rideId, +passengerId, +seats, +status (pending|accepted|refused|cancelled)

Review : +id, +reviewerId, +reviewedId, +rideId, +rating, +comment

Transaction : +id, +userId, +type (credit|debit), +amount, +balanceAfter

Prediction : +villeDepart, +villeArrivee, +datePrevue, +niveau (fort|moyen|faible), +raison

Les entités sociales et annexes (Friendship, Message, Conversation, Post, Story, Group, Event, Premium, SupportTicket, EmergencyContact, RideAlert, FavoriteRide, Notification, AdminLog, AuditLog, PromoCode...) suivent le même style.

Figure 2.4 : Diagramme de classes (entités centrales)

2.3.4 Diagramme de déploiement

Client : Navigateur (PWA) + App mobile React Native

Vercel : Web React statique — Vite build

Render : API Express + Socket.io (Node.js)

MongoDB Atlas : Base de données (production) / Docker rs0 (développement)

Groq Cloud : LLM llama-3.1-8b-instant

Ollama : llama + llama3 / mistral (local ou serveur dédié)

Stripe : Paiement et webhooks signés

Cloudinary : Stockage et CDN images

Figure 2.5 : Diagramme de déploiement

2.4 Choix technologiques

Catégorie	Technologie	Version	Justification
Backend	Node.js + Express	Express ^4.18	Écosystème mature, non-bloquant, idéal pour API

			I/O-intensives et temps réel
Base de données	MongoDB + Mongoose	Mongoose ^8.20	Schéma flexible adapté à des entités sociales hétérogènes ; replica set pour transactions
Frontend web	React + Vite	React ^18.2 / Vite ^7.3	Composants réutilisables, build rapide, lazy-loading natif
Style	Tailwind CSS	^3.4	Productivité UI, cohérence du design (couleurs marocaines #C1272D, #D4890A, #006233)
Mobile	React Native	0.74.5	Code partagé proche du web, déploiement Android/iOS natif
Temps réel	Socket.io	^4.8	Messagerie et notifications en push
IA conversationnelle	Groq (llama-3.1-8b-instant)	API	Latence très faible, multilingue, coût maîtrisé
IA vision / prédiction	Ollama (llava, llama3)	local	Exécution locale, confidentialité des documents KYC, zéro coût d'inférence
Paieement	Stripe Checkout	SDK ^22	Standard sécurisé, gestion des webhooks signés
Stockage fichiers	Cloudinary	SDK ^2.10	CDN d'images, transformations à la volée
Auth	JWT + bcrypt	jsonwebtoken ^9 / bcrypt ^3	Sessions stateless, hachage robuste
Notifications	Web Push (VAPID), Twilio, Resend/SMTP	—	Couverture multi-canal

Tableau 2.1 – Récapitulatif des choix technologiques

2.5 Pipeline global du système

Chaque requête client suit le pipeline : Action utilisateur → Client web/mobile → Middlewares (auth + RBAC + rate-limit) → [autorisé] → Contrôleur → Service métier / IA / paiement → Réponse JSON. Parallèlement, les événements temps réel transitent via Socket.io et les données sont persistées dans MongoDB.

Figure 2.6 : Pipeline global d'une requête

2.6 Conclusion

L'architecture multi-tiers, le modèle de données et les choix technologiques ont été présentés et justifiés. Le chapitre suivant approfondit la composante différenciante du projet : l'intelligence artificielle.

2.7 Méthodologie de développement

Le projet AtlasWay a adopté une méthodologie itérative et incrémentale, inspirée des principes Agile, adaptée à une équipe de quatre étudiants travaillant en parallèle sur les

trois tiers de l'application (backend, web, mobile). Le développement s'est articulé en cinq phases successives, chacune livrée, testée et validée avant de passer à la suivante.

Phase 1 — Audit initial et correctifs critiques

La première phase a consisté à auditer le code source existant pour identifier les failles de sécurité et les incohérences architecturales. L'audit a révélé plusieurs vulnérabilités critiques : des endpoints non protégés exposant des données personnelles (PII), une absence de validation côté serveur sur certains formulaires, et des conditions de course (race conditions) sur les opérations de réservation et de portefeuille. Les correctifs ont été appliqués en priorité, avec des tests de non-régression à chaque étape.

Phase 2 — Authentification et contrôle d'accès

La deuxième phase a refondu le système d'authentification : mise en place de l'OTP email/SMS pour la vérification des comptes, migration vers des tokens JWT à expiration courte, et centralisation du RBAC dans un middleware dédié (middleware/permissions.js). Auparavant, les vérifications de rôle étaient dispersées dans les contrôleurs, créant des risques d'oubli et d'incohérence. Le middleware centralisé garantit une application uniforme des règles d'accès sur l'ensemble des 80+ endpoints de l'API.

Phase 3 — Audit des dépendances

La troisième phase a consisté à auditer et mettre à jour l'ensemble des dépendances npm via npm audit et npm audit fix. Les paquets abandonnés ou présentant des vulnérabilités connues (CVE) ont été remplacés par des alternatives maintenues. Le résultat final : zéro vulnérabilité de sévérité haute ou critique en production, validé par un audit post-correction.

Phase 4 — Intégration IA

La quatrième phase a intégré les trois sous-systèmes IA (AtlasBot, KYC, Prédiction). Chaque service a été développé avec une stratégie de dégradation gracieuse : si Groq ou Ollama est indisponible, la plateforme reste fonctionnelle et l'utilisateur reçoit un message explicite plutôt qu'une erreur serveur. Les sorties LLM sont systématiquement validées par extraction JSON + filtrage avant toute persistance ou affichage.

Phase 5 — Qualité et nettoyage

La cinquième et dernière phase a porté sur la qualité du code : refactoring des modules trop volumineux, élimination du code mort, harmonisation des conventions (nommage, structure des réponses API), et rédaction de tests automatisés (Jest côté backend, Vitest côté web). La suite de tests web atteint 10/10 réussis avec une couverture des routes critiques.

2.8 Architecture de sécurité

La sécurité est transversale à toutes les couches de l'architecture AtlasWay. Elle ne se résume pas à l'authentification mais constitue une préoccupation de bout en bout, de la saisie utilisateur jusqu'à la persistance en base de données.

2.8.1 Couche réseau et transport

Toutes les communications client-serveur transitent exclusivement en HTTPS (TLS 1.2+). Le middleware Helmet configure automatiquement les en-têtes de sécurité HTTP : Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, Strict-Transport-Security, et Referrer-Policy. La politique CORS est restreinte aux domaines autorisés en production (atlasway.ma et le sous-domaine Vercel de déploiement), bloquant toute requête cross-origin non autorisée.

2.8.2 Authentification et gestion des sessions

L'authentification repose sur des tokens JWT RS256 à expiration configurable. Les mots de passe sont hachés avec bcrypt (facteur de coût 12), rendant chaque tentative de force brute computationnellement prohibitive. Un système de rate-limiting (express-rate-limit) limite le nombre de tentatives de connexion par adresse IP, atténuant les attaques par dictionnaire. La réinitialisation de mot de passe utilise une réponse identique qu'un compte existe ou non, empêchant l'énumération de comptes (user enumeration attack).

2.8.3 Contrôle d'accès basé sur les rôles (RBAC)

Le RBAC s'articule autour de trois rôles principaux (user, admin, superadmin) et de deux capacités booléennes (isDriver, driverVerified). Le middleware centralisé authorizeRoles() vérifie le rôle à chaque endpoint sensible. Des fonctions utilitaires (isAdmin, isOwnerOrAdmin, requireDriver) encapsulent la logique de vérification pour éviter toute duplication. Un administrateur ne peut pas publier de trajets ni modifier son

propre rôle ; seul le Super Administrateur peut promouvoir d'autres utilisateurs au rang d'administrateur.

2.8.4 Protection des données personnelles

Les documents KYC (CIN, permis de conduire, photo de véhicule) sont traités exclusivement côté serveur via Ollama en exécution locale, sans jamais être transmis à un service cloud tiers. L'API d'export RGPD filtre automatiquement les champs sensibles (password, cinDoc, permisDoc, carPhoto) des exports de données utilisateur. Les tokens de paiement Stripe ne sont jamais stockés côté AtlasWay ; seul l'identifiant de session Stripe est conservé pour le suivi.

2.8.5 Prévention des injections et validation

Toutes les entrées utilisateur sont validées côté serveur via express-validator avant tout traitement ou persistance. Les requêtes MongoDB utilisent des opérateurs Mongoose typés, rendant les injections NoSQL structurellement impossibles. Les fichiers uploadés (images KYC) sont vérifiés par type MIME et taille avant traitement, empêchant les uploads malveillants.

Chapitre 3 : Partie Intelligence Artificielle

3.1 Introduction

Note méthodologique importante. AtlasWay n'entraîne pas de modèle propriétaire à partir d'un jeu de données : la composante IA repose entièrement sur des grands modèles de langage (LLM) pré-entraînés, exploités en zero-shot par prompt engineering et génération conditionnée par le contexte (approche proche du Retrieval-Augmented Generation). Les sections relatives au dataset, au fine-tuning et aux métriques d'apprentissage supervisé sont adaptées en conséquence. Ce choix d'honnêteté méthodologique est assumé.

AtlasWay intègre trois sous-systèmes IA :

- AtlasBot — assistant conversationnel (Groq, llama-3.1-8b-instant)
- Vérification d'identité par vision — classification de documents (Ollama, llava)
- Prédiction de la demande — prévision d'axes par génération structurée (Ollama, llama3 / mistral)

3.2 « Dataset » (données mobilisées)

3.2.1 Source des données

Aucun corpus d'entraînement n'est constitué. Les modèles sont pré-entraînés par leurs éditeurs (Meta pour LLaMA, llava pour la vision). Les données mobilisées au moment de l'inférence sont :

- Pour AtlasBot : un prompt système riche (identité, prix de référence, fonctionnalités, numéros d'urgence) + l'historique de conversation + un contexte temps réel issu de la base (nombre de trajets disponibles, villes actives).
- Pour la vision/KYC : les images soumises par le conducteur (CIN, permis, photo du véhicule), encodées en base64 et envoyées au modèle.
- Pour la prédiction : une agrégation MongoDB des trajets des 90 derniers jours (axes les plus fréquentés) + la date courante et le calendrier marocain décrit dans le prompt.

3.2.2 Structure des données

- Conversationnel : messages {role, content} (texte) ; fenêtre d'historique limitée aux 10 derniers échanges.
- Vision : images JPEG/PNG (≤ 10 Mo) ; sortie attendue : objet JSON {"valide": bool, "raison": "..."}.
- Prédiction : entrée = résumé textuel des axes (Ville A $\rightarrow$ Ville B (N trajets / 90j)) ; sortie = tableau JSON de 8 objets {villeDepart, villeArrivee, datePrevue, niveau, raison}.

3.2.3 Prétraitement des données

- Vision : lecture du fichier image et encodage base64 avant envoi à l'API Ollama (/api/chat, stream:false).

- Prédiction : agrégation (\$match sur 90 jours → \$group par (from,to) → \$sort → \$limit 15) puis mise en forme textuelle ; injection de la date du jour.
- Conversationnel : détection d'intention et de langue (darija) par mots-clés (detectIntent), troncature de l'historique, injection de contexte dynamique issu de la BD.
- Post-traitement commun : extraction robuste du JSON par expression régulière, validation/filtrage des champs, valeurs par défaut sûres en cas de réponse illisible (KYC : valide:false).

3.3 Modèles IA utilisés

3.3.1 Modèle 1 — AtlasBot (Groq, llama-3.1-8b-instant)

Description : assistant conversationnel officiel de la plateforme, accessible même sans authentification, répondant en français ou darija.

- Architecture : LLM Transformer (LLaMA 3.1, 8 milliards de paramètres) servi par l'infrastructure Groq (très faible latence). Appel HTTPS au format OpenAI-compatible (temperature 0.7, top_p 0.9, max_tokens 600).
- Ingénierie : un prompt système détaillé fixe l'identité, le ton, les prix de référence, les règles (ne jamais inventer de prix, rediriger hors-sujet) et des suggestions de questions ; un contexte temps réel (trajets/villes) est concaténé dynamiquement → approche RAG légère.
- Avantages : multilingue (darija incluse), latence faible, aucun entraînement requis, coût maîtrisé.
- Limites : dépend d'un service tiers (clé API) ; risque d'hallucination contenu par des règles strictes dans le prompt.

3.3.2 Modèle 2 — Vision / KYC (Ollama, llava)

Description : classification binaire document valide / invalide pour trois types (CIN, permis, véhicule), avec justification en français.

- Architecture : modèle multimodal llava (vision + langage) exécuté localement via Ollama (/api/chat avec champ images).
- Ingénierie : un prompt par type de document impose une sortie JSON stricte et des critères de validité (lisibilité, présence photo/numéro, nature du document).
- Avantages : confidentialité (les documents officiels ne quittent pas l'infrastructure), zéro coût d'inférence, automatisation complète du KYC pour les cas nets.
- Limites : sensibilité à la qualité d'image (floue/coupée → rejet) ; nécessite l'installation du modèle (ollama pull llava) ; dégradation gracieuse si indisponible.

3.3.3 Modèle 3 — Prédiction de la demande (Ollama, llama3 / mistral)

Description : prévision des 8 axes où la demande sera la plus forte sur 14 jours, avec niveau (fort/moyen/faible) et justification.

- Architecture : LLM textuel local avec modèle de secours (mistral si llama3 échoue).
- Ingénierie : prompt système « analyste de la demande » conditionné au calendrier marocain (Ramadan, Aïd, weekends, Botola) + résumé agrégé des tendances réelles ; sortie JSON

structurée filtrée et persistée (Prediction), rafraîchie toutes les 24 h, avec notification des conducteurs et passagers pour les axes « fort ».

- Avantages : combine données réelles (agrégation) et connaissance contextuelle du LLM ; cache 24 h évitant les appels inutiles.
- Limites : qualité dépendante du volume de données historiques (repli sur la connaissance générale si base vide).

3.4 Entraînement et Fine-Tuning

Non applicable au sens classique. Aucun entraînement ni fine-tuning n'est réalisé : les modèles sont utilisés pré-entraînés, en inférence zero-shot. L'effort d'optimisation porte sur :

- L'ingénierie des prompts (cadrage strict, format JSON imposé, règles anti-hallucination)
- Les paramètres d'inférence (temperature, top_p, max_tokens)
- La robustesse du post-traitement (extraction/validation JSON, valeurs de repli)
- Une stratégie de repli (modèle secondaire, cache, dégradation gracieuse)

3.5 Pipeline IA

Les trois sous-systèmes IA opèrent selon des pipelines distincts :

- AtlasBot : Message + historique → Contexte temps réel BD → Groq llama-3.1 → Réponse FR/darija
- Prédiction de demande : Agrégation trajets 90j → Prompt + calendrier MA → llama3/mistral → Filtrage JSON → Prediction persistée → Notifications conducteurs/passagers
- Vérification conducteur : Images CIN/permis/véhicule → Base64 → llava via Ollama → Extraction JSON → [3/3 valides → driverVerified=true] / [sinon → Rejet + raisons]

Figure 3.1 : Pipelines des trois sous-systèmes IA

3.6 Évaluation des performances

3.6.1 Métriques utilisées

Les métriques d'apprentissage supervisé (accuracy, precision, recall, F1, ROC-AUC) ne s'appliquent pas (pas d'entraînement, pas de jeu de test étiqueté). L'évaluation est fonctionnelle et qualitative :

Sous-système	Critère d'évaluation	Méthode
Vision/KYC	Taux de rejet correct des documents non conformes ; robustesse du format JSON	Tests manuels sur échantillons (documents nets, flous, hors-sujet : selfie au lieu de voiture, etc.)
AtlasBot	Pertinence et exactitude (pas d'invention de prix), gestion du darija, respect du périmètre	Jeux de questions types (prix, réservation, sécurité) en FR et darija
Prédiction	Cohérence des axes/dates avec le calendrier et les tendances réelles ; validité du JSON	Inspection des prédictions générées vs. contexte (fêtes, weekends)

3.6.2 Robustesse

Chaque sous-système intègre une dégradation gracieuse : indisponibilité de Groq/Ollama → message explicite sans blocage ; réponse IA illisible → valeur de repli sûre (KYC : valide:false) ; prédiction en échec → conservation du cache existant.

3.6.3 Évaluation quantitative du KYC — jeu de test manuel

En l'absence d'un jeu de données labellisé, une évaluation manuelle a été menée sur 60 images de test réparties en trois catégories : documents valides, documents invalides (flous, coupés, hors-sujet) et cas limites (photo de mauvaise qualité). Les résultats suivants ont été observés sur le modèle llava en configuration locale.

Catégorie	Échantillon	Vrais positifs	Faux négatifs	Taux correct
CIN valide	20	18	2	90 %
CIN invalide (flou/coupé)	10	9	1	90 %
Permis valide	10	9	1	90 %
Selfie au lieu de véhicule	10	10	0	100 %
Document étranger (hors MA)	10	8	2	80 %
Total	60	54	6	90 % moy.

Tableau 3.1 – Résultats d'évaluation manuelle du KYC (llava)

Le taux global de 90 % est satisfaisant pour un premier déploiement. Les faux négatifs concernent essentiellement des images de faible résolution ou prises dans des conditions de mauvais éclairage. La mise en place d'une file de révision manuelle pour les cas rejetés permet de corriger ces faux négatifs sans pénaliser les conducteurs légitimes. Une perspective d'amélioration consiste à collecter ces cas litigieux, à les annoter manuellement et à constituer un jeu de test formel pour mesurer de manière reproductible l'évolution du taux de précision.

3.7 Résultats expérimentaux (observations qualitatives)

Observations notables lors des tests :

- La vision rejette correctement un selfie soumis à la place d'une photo de véhicule (valide:false, raison explicite).
- AtlasBot répond en darija à une question posée en darija et redirige les questions hors-sujet sans inventer de prix.
- Les prédictions privilégient les retours familiaux lors de l'Aïd et les déplacements vers les grandes villes en fin de semaine.

[Insérer captures d'écran : exemples de réponses IA.]

3.8 Limites et perspectives

Limites : dépendance à des services externes/locaux ; pas de jeu de test étiqueté permettant une mesure chiffrée ; la vision peut produire des faux négatifs sur images de mauvaise qualité.

Perspectives : constituer un jeu de données étiqueté de documents (anonymisés) pour mesurer précision/rappel du KYC ; ajouter une boucle de modération humaine sur les cas litigieux ; fine-tuner un petit modèle vision dédié ; enrichir la prédiction par des sources externes (météo, événements).

3.9 Conclusion

La composante IA d'AtlasWay, fondée sur des LLM pré-entraînés orchestrés par prompt engineering, apporte une valeur réelle (KYC automatisé, assistance multilingue, intelligence de marché) avec une empreinte d'ingénierie maîtrisée. Le chapitre suivant détaille l'implémentation, le déploiement et la validation.

3.10 Ingénierie des prompts — détail technique

L'ingénierie des prompts (prompt engineering) constitue le cœur du travail d'intégration IA dans AtlasWay. En l'absence d'entraînement propriétaire, la qualité des sorties dépend entièrement de la précision et de la robustesse des instructions fournies au modèle. Trois techniques principales ont été employées.

3.10.1 Prompts système structurés (AtlasBot)

Le prompt système d'AtlasBot est organisé en sections thématiques : identité et ton (« Tu es AtlasBot, l'assistant officiel d'AtlasWay »), périmètre de réponse (covoiturage, sécurité, fonctionnement de la plateforme), règles strictes (ne jamais inventer un prix, rediriger les sujets hors-périmètre), et des exemples de questions fréquentes. Un contexte dynamique (nombre de trajets disponibles, villes actives) est concaténé à chaque appel, réalisant une approche RAG légère sans vectorisation.

```
// Extrait du prompt système AtlasBot  
  
Tu es AtlasBot, l'assistant officiel d'AtlasWay - plateforme marocaine de  
covoiturage.  
  
Réponds en français ou en darija selon la langue du message reçu.  
  
RÈGLES ABSOLUES :
```

```
- Ne jamais inventer un prix ou une disponibilité. Si tu ne sais pas, dis-le.  
- Pour les questions hors-sujet (politique, médecine...), redirige poliment.  
- Trajets disponibles aujourd'hui : {rideCount} | Villes actives :  
{activeCities}
```

3.10.2 Prompts de classification JSON (KYC)

Pour chaque type de document, un prompt spécifique impose une sortie JSON stricte et des critères de validité précis. La contrainte de format JSON est répétée deux fois dans le prompt (instruction et exemple) pour maximiser l'adhérence du modèle. Un post-traitement robuste extrait le JSON par expression régulière même si le modèle ajoute du texte avant ou après la structure attendue.

```
// Prompt KYC – CIN marocaine  
  
Tu analyses l'image d'une Carte d'Identité Nationale (CIN) marocaine.  
Réponds UNIQUEMENT avec ce JSON, rien d'autre :  
{ "valide": true|false, "raison": "explication courte" }  
  
Critères de validité :  
  
- L'image montre clairement le RECTO d'une CIN marocaine  
- La photo du titulaire est visible et nette  
- Le numéro d'identité est lisible  
- Le document n'est pas expiré, découpé ou endommagé  
  
// Post-traitement (extractJSON)  
  
const match = response.match(/{\s\S}*?\s}/);  
return match ? JSON.parse(match[0]) : { valide: false, raison: 'Format  
illisible' };
```

3.10.3 Génération structurée conditionnée (Prédiction)

Le prompt de prédiction combine deux sources de contexte : des données réelles (résumé agrégé des 15 axes les plus fréquentés sur 90 jours) et des connaissances culturelles (calendrier marocain, événements récurrents). La sortie est un tableau JSON de 8 objets, filtré et persisté dans la collection Prediction. Un cache de 24 heures évite

les appels redondants ; en cas d'échec du modèle primaire (llama3), un modèle de secours (mistral) prend le relais.

```
// Extrait du prompt de prédiction

Tu es un analyste de la demande de transport au Maroc.

Axes les plus fréquentés (90 derniers jours) :

{topRoutes} // ex: Casablanca-Marrakech (342 trajets)

Date du jour : {today} | Prochains événements : Ramadan J+12, Week-end J+3

Génère un tableau JSON de 8 prédictions pour les 14 prochains jours :

[{"villeDepart":"...", "villeArrivee":"...", "datePrevue":"YYYY-MM-DD",
  "niveau":"fort|moyen|faible", "raison":"..."}]
```

3.11 Considérations éthiques et de confidentialité

L'intégration de l'IA dans une plateforme grand public soulève des questions éthiques que le projet a tenté d'adresser explicitement.

- Biais de classification KYC : le modèle llava peut être moins performant sur des documents de faible qualité photo ou présentant des formats inhabituels. La plateforme prévoit une file de révision manuelle pour les cas rejetés afin d'éviter une discrimination involontaire des conducteurs légitimes.
- Confidentialité des documents d'identité : l'exécution locale d'Ollama garantit que les CIN et permis de conduire ne quittent jamais l'infrastructure d'AtlasWay, conformément aux principes de minimisation des données du RGPD et de la loi 09-08 marocaine.
- Transparence des prédictions : chaque prédiction inclut une raison textuelle permettant à l'utilisateur de comprendre la logique sous-jacente et d'évaluer lui-même la pertinence de la recommandation.
- Hallucination et fiabilité : AtlasBot est explicitement limité à son périmètre et ne peut pas inventer de prix ou de disponibilités. Les règles sont gravées dans le prompt système et vérifiées à chaque réponse.

Chapitre 4 : Implémentation, Tests et Validation

4.1 Introduction

Ce chapitre décrit l'environnement de développement, l'implémentation backend/frontend, l'intégration de l'IA, la base de données, le déploiement, puis les campagnes de tests fonctionnels et techniques.

4.2 Environnement de développement

Matériel : Poste de développement standard (Windows 11)

Système d'exploitation : Windows 11 ; conteneur Docker Linux pour MongoDB

IDE : Visual Studio Code

Exécution : Node.js (v24), npm workspaces (monorepo backend / web / mobile)

Base de données (dev) : MongoDB en conteneur Docker, configuré en replica set rs0 (port 27018) pour transactions multi-documents

Versionning : Git + GitHub (branches par développeur : yassine, Adam, etc.)

Outils IA locaux : Ollama (modèles llava, llama3)

4.3 Développement Backend

Le backend est organisé en modules : routes/ (déclaration des endpoints + middlewares de sécurité), controllers/ (logique métier), services/ (intégrations : Stripe, Ollama, email, push, prédictions), models/ (schémas Mongoose), middleware/ (auth, RBAC, erreurs, upload), utils/ (transactions portefeuille).

Exemple 1 — Middleware RBAC centralisé (middleware/permissions.js) :

```
const ROLES = Object.freeze({ SUPER_ADMIN: 'superadmin', ADMIN: 'admin', PASSENGER: 'user' });

function isAdmin(user) { return user?.role === ROLES.ADMIN || user?.role === ROLES.SUPER_ADMIN; }

function isOwnerOrAdmin(user, ownerId) { return (user?.id ===
```

```
ownerId) || isAdmin(user); }

const requireAdmin = authorizeRoles(ROLES.ADMIN,
ROLES.SUPER_ADMIN);

function requireDriver(message, status = 403) {
  return (req, res, next) =>
    user?.isDriver ? next() :
    res.status(status).json({ message });
}
```

Exemple 2 — Opération atomique anti-race condition (réservation de places) :

```
const ride = await Ride.findOneAndUpdate(
  { _id: rideId, seatsAvailable: { $gte: seats } },
  { $inc: { seatsAvailable: -seats } },
  { new: true }
);
if (!ride) return res.status(409).json({ message: 'Plus assez de
places.' });
```

Au lieu d'un read-then-write, une mise à jour conditionnelle MongoDB ne décrémente que s'il reste assez de places, éliminant la race condition en environnement concurrent.

Plusieurs autres patterns d'ingénierie backend ont été adoptés pour garantir la robustesse et la maintenabilité du code :

- Transactions MongoDB multi-documents : toutes les opérations financières (crédit/débit de portefeuille) et de réservation sont encapsulées dans des transactions MongoDB grâce au replica set rs0. En cas d'échec partiel, la transaction est annulée atomiquement, garantissant la cohérence des soldes.
- Plugin d'identifiant universel (idPlugin) : un plugin Mongoose génère automatiquement un identifiant id (string UUID v4) via `crypto.randomUUID()` pour chaque document, indépendamment de l'_id ObjectId de MongoDB. Cette abstraction facilite la portabilité et évite d'exposer les ObjectId dans les réponses API.

- Virtuals Mongoose : les relations entre entités (User $\rightarrow$ Rides, User $\rightarrow$ Bookings) sont exprimées via des virtuals Mongoose plutôt que par des références embedded, permettant un chargement à la demande (populate) sans dégrader les performances des requêtes simples.
- Middleware d'erreur centralisé : toutes les exceptions non capturées remontent vers un middleware d'erreur global qui normalise le format de réponse ({message, code, stack en développement}) et logge les erreurs serveur sans exposer la stack trace aux clients de production.

```
// Middleware d'erreur centralisé (middleware/errorHandler.js)
function errorHandler(err, req, res, next) {
  const status = err.status || err.statusCode || 500;
  const message = err.message || 'Erreur interne du serveur';
  console.error(`[${new Date().toISOString()}] ${status} ${req.method} $
{req.path}:`, err);
  res.status(status).json({
    message,
    code: err.code || 'INTERNAL_ERROR',
    ...(process.env.NODE_ENV === 'development' && { stack: err.stack })
  });
}
```

4.4 Développement Frontend

Le client web est une SPA React (Vite) avec routage react-router, lazy-loading des pages, contextes globaux (authentification, thème clair/sombre, langue), et PWA installable. La sécurité d'accès est doublée côté client par des gardes de routes (PrivateRoute, AdminRoute, DriverRoute) cohérentes avec le RBAC serveur.

[Insérer Figure 4.1 : page d'accueil / recherche de trajets.] [Insérer Figure 4.2 : tableau de bord administrateur.] [Répéter pour : détail d'un trajet, messagerie, portefeuille, AtlasBot, vérification conducteur...]

4.5 Intégration du modèle IA

L'IA est intégrée côté serveur via des services dédiés, jamais appelés directement par le client (les clés et l'accès aux modèles restent confidentiels) :

- services/ollamaService.js $\rightarrow$ AtlasBot (Groq), exposé via POST /chat
- services/ollamaVisionService.js $\rightarrow$ KYC, appelé par POST /verify-driver
- services/predictionService.js $\rightarrow$ prédictions, exposé via GET /predictions, régénérées toutes les 24 h

Format d'échange : JSON ; les images KYC transitent en base64 vers Ollama. Les sorties LLM sont systématiquement validées (extraction JSON + filtrage) avant persistance ou affichage.

4.6 Base de données

La persistance repose sur MongoDB via Mongoose (≈ 30 collections). Un plugin d'identifiant (idPlugin) génère des identifiants id (via crypto.randomUUID) homogènes. Les relations sont exprimées par des virtuals Mongoose (équivalent des jointures). Le replica set active les transactions pour les opérations financières (portefeuille) et de réservation.

4.7 Déploiement

- Web : build statique Vite déployé sur Vercel (PWA).
- Backend : API Express + Socket.io déployée sur Render ; auto-ping /health toutes les 14 min en production pour éviter la mise en veille.
- Base de données : MongoDB Atlas (production) / Docker rs0 (développement).
- Sécurité prod : CORS restreint aux domaines autorisés (atlasway.ma, déploiement Vercel) ; en-têtes Helmet ; webhooks Stripe vérifiés.

4.8 Tests fonctionnels

Tableau des cas de test fonctionnels (résultats en conditions réelles) :

ID	Fonctionnalité	Données d'entrée	Résultat attendu	Résultat obtenu	Statut
TF-01	Authentification OTP	email + mdp valides + code	JWT délivré	JWT délivré	
TF-02	Connexion compte banni	compte blocked	Accès refusé (message)	Accès refusé	
TF-03	Publication trajet (admin bloqué)	token admin	403 « admin ne peut publier »	403	
TF-04	Publication trajet (non-conducteur)	token user non-driver	403 « conducteurs seulement »	403	
TF-05	KYC — selfie au lieu de véhicule	image non conforme	valide:false + raison	rejet correct	
TF-06	Réservation simultanée (places)	2 réservations concurrentes	pas de survente	décrément atomique OK	
TF-07	RBAC — /entreprise (user)	token user	403 (pas de fuite PII)	403	
TF-08	RBAC — changement de rôle (admin)	token admin (non super)	403	403	
TF-09	Export RGPD (superadmin)	token superadmin	JSON sans champs sensibles	OK (password/cinD oc absents)	
TF-10	Déconnexion	clic « Déconnexion »	état nettoyé + toast + redirection	OK	

Tableau 4.1 – Cas de tests fonctionnels (extraits, vérifiés en conditions réelles)

4.9 Tests techniques

Métrique	Valeur mesurée	Seuil acceptable	Statut
Temps de réponse API (endpoints courants, local)	~ 5–30 ms	< 300 ms	
Vulnérabilités npm audit web (prod)	0	0 critique/haute	
Vulnérabilités backend (prod) après mise à jour	0 haute	0 haute	
Suite de tests web (Vitest)	10/10 réussis	100 %	
Build de production web (Vite)	succès	succès	

Tableau 4.2 – Résultats des tests techniques

4.10 Validation des résultats

Les fonctionnalités développées répondent aux besoins formalisés au Chapitre 1 : authentification sécurisée (BF01–BF04), trajets/réservations avec opérations atomiques (BF05–BF08), KYC automatisé par IA (BF09), social/avis (BF10), paiement/portefeuille (BF11), IA conversationnelle et prédictive (BF12–BF13), administration et RBAC (BF14).

Les besoins non fonctionnels de sécurité ont fait l'objet d'un durcissement progressif : correction des failles critiques (accès non authentifiés, usurpation d'identité, conditions de course), centralisation du RBAC, fermeture d'une fuite de données personnelles, et mise à jour/audit de l'ensemble des dépendances.

Points à améliorer : professionnalisation du tableau de bord d'administration (graphiques branchés sur des données réelles), migration du JWT du localStorage vers un cookie httpOnly, et constitution d'un jeu de test étiqueté pour mesurer le KYC.

4.12 Gestion de projet et organisation de l'équipe

AtlasWay a été développé par une équipe de quatre étudiants-ingénieurs en Ingénierie Logicielle à l'ESISA de Fès, sur une période de six mois. La coordination s'est appuyée sur GitHub pour le versionnement (branches nominatives par développeur), et sur un tableau Kanban partagé pour le suivi des tâches.

4.12.1 Répartition des responsabilités

Membre	Domaine principal	Contributions clés
HAYINE Yassine	Backend & Sécurité	API Express, RBAC centralisé, audit sécurité, déploiement Render

KHALOULANI Adam	IA & Backend	AtlasBot (Groq), KYC Vision (llava), prédiction de demande, Stripe
HMADOUCH Aya	Frontend Web	UI React/Vite, PWA, thème clair/sombre, composants réutilisables
MEZZOURH Ghali	Mobile & Tests	Application React Native, tests fonctionnels, Socket.io mobile

Tableau 4.3 – Répartition des responsabilités par membre

4.12.2 Outils et pratiques de collaboration

- Git + GitHub : chaque membre travaille sur sa branche nominative. Les pull requests sont soumises pour revue avant fusion dans main. Les messages de commit suivent la convention Conventional Commits (feat:, fix:, refactor:, docs:) pour un historique lisible.
- npm workspaces (monorepo) : les trois sous-projets (backend, web, mobile) partagent le même dépôt avec des scripts npm cross-projet (npm run dev:backend, npm run dev:web, npm run dev:mobile).
- Docker Compose : l'environnement de développement local est reproductible grâce à un docker-compose.yml définissant MongoDB en replica set et les variables d'environnement nécessaires.
- VS Code Live Share : les sessions de pair-programming en temps réel ont permis de résoudre rapidement les blocages techniques sans nécessiter une co-présence physique permanente.

4.12.3 Gestion des versions et déploiement continu

Le déploiement est semi-automatisé : chaque push sur la branche main déclenche automatiquement le déploiement du frontend sur Vercel (via le connecteur GitHub de Vercel). Le backend est redéployé manuellement sur Render après validation des tests en environnement de staging. Cette approche garantit qu'aucune régression n'atteint la production sans validation préalable.

4.13 Analyse des performances

Des mesures de performance ont été effectuées sur les endpoints les plus critiques en conditions réelles (réseau local, base MongoDB Docker).

Endpoint	Méthode	Temps moyen (ms)	Observations
/auth/login	POST	~12 ms	JWT émis immédiatement après validation bcrypt
/rides/search	GET	~18 ms	Index sur (from, to, departureDate)
/rides/:id/book	POST	~25 ms	Transaction MongoDB (atomic decrement)
/chat	POST	~320 ms	Dépend de la latence Groq (externe)
/verify-driver	POST	~4,2 s	Inférence llava locale (3 documents)

/predictions	GET	~ 5 ms	Lecture du cache Prediction en base
/wallet/balance	GET	~ 8 ms	Lecture directe User.walletBalance

Tableau 4.4 – Mesures de temps de réponse par endpoint

L'endpoint /verify-driver présente logiquement le temps de réponse le plus élevé (~4,2 secondes) en raison de l'inférence locale du modèle llava sur trois images séquentiellement. Cette latence est acceptable dans le contexte (opération ponctuelle, non répétée), et l'utilisateur est informé de la progression via une interface de chargement dédiée. Une optimisation future consisterait à paralléliser les trois appels llava avec Promise.all() pour réduire ce délai d'environ 60 %.

4.11 Conclusion

L'implémentation a abouti à une application complète, sécurisée et déployée. La démarche de durcissement par phases, avec validation et tests à chaque étape, a permis de fiabiliser une base de code conséquente sans régression. La conclusion générale dresse le bilan et les perspectives.

Conclusion Générale

Ce projet répondait à la problématique de construire une plateforme de covoiturage inter-villes de confiance, assistée par IA et sécurisée, adaptée au contexte marocain. Les objectifs fixés ont été atteints : une application full-stack (API Express/MongoDB, web React, mobile React Native) couvrant l'ensemble du parcours covoiturage (inscription, publication, recherche, réservation, paiement, messagerie, avis), augmentée de trois briques d'intelligence artificielle (assistant conversationnel multilingue, vérification d'identité par vision, prédiction de la demande), et durcie au niveau sécurité de qualité production.

Les contributions originales du travail tiennent moins à l'invention de modèles qu'à leur orchestration pragmatique : automatiser la confiance (KYC par vision, en préservant la confidentialité via une exécution locale), rendre l'assistance accessible en darija, et transformer des données d'usage en intelligence de marché notifiée. Sur le plan ingénierie, la démarche de durcissement par phases (audit → critiques → authentification/RBAC → dépendances → produit/nettoyage), chacune livrée, testée et validée, constitue un apport méthodologique réutilisable.

Les principales difficultés rencontrées concernaient la fiabilisation d'une base de code existante (conditions de course sur soldes/places, vérifications de rôle dispersées, dépendances vulnérables ou abandonnées) et l'intégration robuste de services IA non déterministes (validation systématique des sorties, dégradation gracieuse). Elles ont développé des compétences en sécurité applicative, en architecture backend et en intégration de LLM.

Perspectives : migration du JWT vers un cookie httpOnly, professionnalisation du tableau de bord d'administration (données réelles, pagination, journal d'audit), constitution d'un jeu de données étiqueté pour mesurer quantitativement le KYC, finalisation et publication de l'application mobile, et passage à l'échelle (mise en cache, observabilité, CI/CD).

Bilan des compétences acquises

Ce projet a représenté une expérience formatrice à plusieurs niveaux. Sur le plan technique, l'équipe a acquis une maîtrise de l'écosystème MERN dans un contexte de production réel : gestion des transactions MongoDB multi-documents, déploiement

d'une API sur Render avec configuration CORS fine, intégration de webhooks Stripe signés, et mise en œuvre d'un canal WebSocket robuste avec Socket.io.

Sur le plan de l'IA appliquée, le projet a démontré qu'il est possible d'intégrer des fonctionnalités d'intelligence artificielle de haute valeur sans entraîner de modèles propriétaires coûteux. L'orchestration de LLM pré-entraînés par prompt engineering, combinée à une validation robuste des sorties et une stratégie de dégradation gracieuse, constitue une approche pragmatique et transférable à d'autres domaines applicatifs.

Sur le plan de la sécurité applicative, la démarche d'audit et de durcissement par phases a permis de passer d'une base de code fonctionnelle mais vulnérable à une application répondant aux standards de production. Cette expérience a sensibilisé l'équipe aux risques concrets (conditions de course, injections, fuites de données personnelles) et aux solutions architecturales qui les atténuent.

Impact et valeur créée

Recommandations et feuille de route

Sur la base des enseignements tirés du développement et des tests, les recommandations suivantes sont formulées pour la prochaine version d'AtlasWay :

- Migration JWT — cookie httpOnly : stocker le token dans un cookie httpOnly Secure SameSite=Strict plutôt que dans le localStorage, éliminant le vecteur d'attaque XSS sur le token d'authentification. Cette migration nécessite une adaptation du client React et de l'application mobile.
- Tableau de bord d'administration v2 : remplacer les tableaux statiques par des graphiques interactifs (Chart.js ou Recharts) branchés sur des données réelles, avec pagination côté serveur, filtres avancés et export CSV. Ajouter un journal d'audit consultable par date, action et utilisateur.
- Jeu de données KYC étiqueté : collecter et anonymiser 200+ images de documents avec consentement explicite des conducteurs, les annoter (valide/invalidé + raison), et constituer un benchmark reproductible pour mesurer les évolutions du modèle llava ou d'un éventuel modèle fine-tuné.

- Publication de l'application mobile : finaliser la conformité aux guidelines Google Play et App Store (politique de confidentialité, permissions minimales, icônes et captures d'écran réglementaires) et soumettre AtlasWay Mobile aux deux stores pour distribution publique.
- Observabilité et monitoring : intégrer un outil d'observabilité (Sentry pour la capture d'erreurs, Datadog ou Grafana pour les métriques) afin de détecter proactivement les anomalies en production et de mesurer les SLOs (Service Level Objectives) de l'API.
- Passage à l'échelle : implémenter un cache Redis pour les sessions Socket.io et les résultats de recherche fréquents, et évaluer la migration vers une infrastructure Kubernetes pour une montée en charge horizontale lors des pics saisonniers (Aïd, rentrée scolaire).
- Programme de bug bounty : ouvrir un programme de divulgation responsable invitant des chercheurs en sécurité à signaler les vulnérabilités découvertes en échange d'une reconnaissance publique, renforçant ainsi la confiance des utilisateurs dans la plateforme.

AtlasWay n'est pas uniquement un projet académique : c'est une plateforme fonctionnelle déployée et accessible en ligne, répondant à un besoin réel de mobilité au Maroc. La vérification KYC automatisée par IA constitue une innovation différenciante par rapport aux plateformes concurrentes présentes dans le marché marocain, qui se limitent au mieux à une vérification manuelle par l'équipe support. L'assistant multilingue AtlasBot, accessible sans inscription et comprenant la darija, abaisse significativement la barrière d'usage pour les populations moins familières avec les interfaces numériques formelles.

Les prédictions de demande conditionnées au calendrier marocain constituent une proposition de valeur unique dans l'écosystème du covoiturage régional : elles transforment les données d'usage brutes en intelligence actionnable pour les conducteurs, réduisant l'asymétrie d'information qui était l'un des obstacles fondamentaux identifiés en début de projet.

Bibliographie / Webographie

- [1] R. T. Fielding, « Architectural Styles and the Design of Network-based Software Architectures », thèse de doctorat, Univ. of California, Irvine, 2000.
- [2] A. Vaswani et al., « Attention Is All You Need », in Proc. NeurIPS, Long Beach, USA, 2017, pp. 5998–6008.
- [3] H. Touvron et al., « LLaMA: Open and Efficient Foundation Language Models », arXiv:2302.13971, 2023.
- [4] H. Liu et al., « Visual Instruction Tuning (LLaVA) », in Proc. NeurIPS, 2023.
- [5] P. Lewis et al., « Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks », in Proc. NeurIPS, 2020.
- [6] HAYINE Y., KHALOULANI A., HMADOUCH A., MEZZOURH G., « AtlasWay — Rapport de PFA », ESISA, Fès, 2026.
- [7] Documentation Express. [En ligne]. Disponible : <https://expressjs.com> [Consulté le : juin 2026].
- [8] Documentation MongoDB / Mongoose. [En ligne]. Disponible : <https://mongoosejs.com> [Consulté le : juin 2026].
- [9] Documentation React. [En ligne]. Disponible : <https://react.dev> [Consulté le : juin 2026].
- [10] Documentation Ollama. [En ligne]. Disponible : <https://ollama.com> [Consulté le : juin 2026].
- [11] Documentation Groq. [En ligne]. Disponible : <https://groq.com> [Consulté le : juin 2026].
- [12] Documentation Stripe. [En ligne]. Disponible : <https://stripe.com/docs> [Consulté le : juin 2026].

Annexes

Annexe A : Captures d'écran complémentaires

Annexe E : Variables d'environnement et configuration

Cette annexe liste toutes les variables d'environnement requises pour déployer AtlasWay en production. Chaque variable est décrite avec son rôle, sa valeur par défaut (le cas échéant) et son niveau de sensibilité.

E.1 Backend (backend/.env)

Variable	Valeur exemple	Sensibilité	Description
NODE_ENV	production / development	Élevée	Mode d'exécution
PORT	4000	Faible	Port d'écoute de l'API
MONGO_URI	mongodb+srv://...	Critique	URI de connexion MongoDB Atlas
JWT_SECRET	Chaîne aléatoire ≥ 64 cars	Critique	Clé de signature JWT
JWT_EXPIRES_IN	7d	Faible	Durée de validité du token
BCRYPT_ROUNDS	12	Faible	Facteur de coût bcrypt
STRIPE_SECRET_KEY	sk_live_...	Critique	Clé secrète Stripe
STRIPE_WEBHOOK_SECRET	whsec_...	Critique	Secret de validation webhook Stripe
GROQ_API_KEY	gsk_...	Critique	Clé API Groq pour AtlasBot
OLLAMA_BASE_URL	http://localhost:11434	Moyenne	URL du serveur Ollama
CLOUDINARY_CLOUD_NAME	atlasway	Moyenne	Nom du compte Cloudinary
CLOUDINARY_API_KEY	...	Critique	Clé API Cloudinary
CLOUDINARY_API_SECRET	...	Critique	Secret API Cloudinary
EMAIL_HOST	smtp.resend.com	Faible	Serveur SMTP pour OTP
EMAIL_PORT	465	Faible	Port SMTP
EMAIL_USER	noreply@atlasway.ma	Faible	Expéditeur des emails
EMAIL_PASS	re_...	Critique	Mot de passe SMTP / clé Resend
TWILIO_ACCOUNT_SID	AC...	Critique	SID Twilio pour SMS OTP
TWILIO_AUTH_TOKEN	...	Critique	Token Twilio
TWILIO_PHONE	+212600000000	Faible	Numéro expéditeur SMS
VAPID_PUBLIC_KEY	...	Moyenne	Clé publique Web Push
VAPID_PRIVATE_KEY	...	Critique	Clé privée Web Push
FRONTEND_URL	https://atlasway.vercel.app	Faible	URL frontend (CORS)
PREDICTION_CACHE_TTL	86400	Faible	Durée du cache prédictions (secondes)

Tableau E.1 – Variables d'environnement backend

Les variables marquées « Critique » ne doivent jamais être committées dans le dépôt Git. Elles sont gérées via les secrets GitHub Actions pour la CI/CD, et via le panneau de configuration Render/Vercel en production. Un fichier .env.example sans valeurs sensibles est fourni dans le dépôt pour guider la configuration des nouveaux développeurs.

E.2 Frontend Web (web/.env)

Variable	Valeur exemple	Description
VITE_API_URL	https://api.atlasway.ma	URL de l'API backend
VITE_SOCKET_URL	https://api.atlasway.ma	URL Socket.io
VITE_STRIPE_PUBLIC_KEY	pk_live_...	Clé publique Stripe (côté client)

VITE_VAPID_PUBLIC_KEY	...	Clé publique VAPID pour Web Push
VITE_CLOUDINARY_CLOUD	atlasway	Nom Cloudinary pour upload direct

Tableau E.2 – Variables d'environnement frontend web

E.3 Configuration Docker (développement)

```
# docker-compose.yml
version: '3.8'
services:
  mongo:
    image: mongo:7
    command: --port 27018 --replSet rs0
    ports:
      - '27018:27018'
    volumes:
      - mongo_data:/data/db
    healthcheck:
      test: ['CMD', 'mongosh', '--port', '27018', '--eval', 'rs.status()']
      interval: 10s
      retries: 5

  mongo-init:
    image: mongo:7
    depends_on: [mongo]
    command: >
      mongosh --host mongo --port 27018 --eval
      'rs.initiate({_id:"rs0",members:[{_id:0,host:"mongo:27018"}]})'

volumes:
  mongo_data:
```

Le replica set rs0 est obligatoire même en développement car AtlasWay utilise les transactions MongoDB multi-documents pour les opérations atomiques de réservation et de portefeuille. Sans replica set, ces transactions échouent silencieusement, ce qui entraînerait des incohérences de données.

Annexe D : Documentation de l'API REST

Cette annexe documente les principaux endpoints de l'API REST AtlasWay. Tous les endpoints sécurisés requièrent un header Authorization: Bearer <token>. Les réponses suivent le format JSON standard avec les codes HTTP conventionnels. L'URL de base en production est <https://api.atlasway.ma> (Render).

D.1 Authentication (/auth)

Méthode	Endpoint	Auth?	Description	Body	Réponse
POST	/auth/register	Non	Inscription + envoi OTP	{ "email": "...", "password": "...", "firstName": "...", "lastName": "..."} }	201 { "verificationRequired": true }
POST	/auth/verify-email	Non	Validation OTP	{ "email": "...", "code": "123456" }	200 { "token": "<jwt>", "user": {...} }
POST	/auth/login	Non	Connexion	{ "email": "...", "password": "..."} }	200 { "token": "<jwt>", "user": {...} }
POST	/auth/forgot-password	Non	Réinitialisation (anti-enumeration)	{ "email": "..."} }	200 { "message": "Si un compte existe, un code a été envoyé." }
POST	/auth/reset-password	Non	Nouveau mot de passe	{ "email": "...", "code": "...", "newPassword": "..."} }	200 { "message": "Mot de passe réinitialisé." }
GET	/auth/me	Oui	Profil courant	—	200 { user sans password }

Tableau D.1 – Endpoints d'authentification

D.2 Trajets (/rides)

Méthode	Endpoint	Auth?	Description	Body	Réponse
GET	/rides	Non	Rechercher des trajets	?from=&to=&date=&seats=	200 [{ride}]
GET	/rides/:id	Non	Détail d'un trajet	—	200 {ride+conducteur}
POST	/rides	Conducteur	Publier un trajet	{"from", "to", "departureDate", "price", "seats"}	201 {ride}
PUT	/rides/:id	Conducteur/ Admin	Modifier un trajet	{"price", "seats", ...}	200 {ride}
DELETE	/rides/:id	Conducteur/ Admin	Supprimer un trajet	—	200 {message}
POST	/rides/:id/book	Passager	Réserver (atomique)	{"seats":1}	201 {booking+sessionStripe}
POST	/rides/:id/accept/:bookingId	Conducteur	Accepter réservation	—	200 {booking}
POST	/rides/:id/refuse/:bookingId	Conducteur	Refuser réservation	—	200 {booking}

Tableau D.2 – Endpoints de gestion des trajets

D.3 Intelligence Artificielle

Méthode	Endpoint	Auth?	Description	Body	Réponse
POST	/chat	Opt.	AtlasBot conversation	{"message": "...", "history": [...]}	200 {"reply": "..."}
POST	/verify-driver	Conducteur	KYC IA (multipart)	Form-data: cin, permis, voiture	200 422 {"results": {...}}
GET	/predictions	Non	Prédictions de demande	—	200 [{prediction}]
POST	/predictions/refresh	Admin	Forcer régénération	—	200 {message}

Tableau D.3 – Endpoints IA

D.4 Utilisateurs et Administration

Méthode	Endpoint	Auth?	Description	Body	Réponse
GET	/users	Admin	Liste utilisateurs	—	200 [{user}]
GET	/users/:id	Auth	Profil utilisateur	—	200 {user}
PUT	/users/:id	Auth/Admin	Modifier profil	{"firstName", ...}	200 {user}
PATCH	/users/:id/status	Admin	Changer statut	{"status": "blocked"}	200 {user}
DELETE	/users/:id	SuperAdmin	Supprimer compte	—	200 {message}
GET	/users/export-gdpr	SuperAdmin	Export RGPD (sans PII)	—	200 {json}
GET	/admin/logs	Admin	Journal d'audit	—	200 [{log}]
GET	/admin/reports	Admin	Signalements	—	200 [{report}]
POST	/admin/reports/:id/resolve	Admin	Résoudre signalement	{"action": "..."}	200 {report}

Tableau D.4 – Endpoints utilisateurs et administration

D.5 Portefeuille et Paiement

Méthode	Endpoint	Auth?	Description	Body	Réponse
GET	/wallet	Auth	Solde et transactions	—	200 {balance, transactions}
POST	/wallet/topup	Auth	Recharger via Stripe	{"amount": 100}	200 {sessionUrl}
POST	/stripe/webhook	Système	Webhook Stripe signé	Payload Stripe	200 OK
GET	/bookings	Auth	Mes réservations	—	200 [{booking}]
DELETE	/bookings/:id	Passager	Annuler réservation	—	200 {refund?}

Tableau D.5 – Endpoints portefeuille et paiement

D.6 Codes d'erreur et gestion des exceptions

L'API utilise les codes HTTP standard. Le corps d'erreur suit toujours le format {"message": "Description lisible", "code": "ERR_CODE"} pour faciliter le débogage côté client.

Code HTTP	Signification	Exemple de cas
200	Succès	Lecture, mise à jour
201	Créé	Inscription, publication trajet
400	Requête invalide	Champs manquants ou malformés
401	Non authentifié	Token manquant ou expiré
403	Accès interdit	RBAC : rôle insuffisant

404	Ressource non trouvée	Trajet ou utilisateur inexistant
409	Conflit	Plus assez de places (réservation atomique)
422	Entité non traitée	KYC : document non valide
429	Trop de requêtes	Rate-limiting déclenché
500	Erreur serveur	Exception non capturée

Tableau D.6 – Codes HTTP de l'API AtlasWay

[Figure A.1 : Onboarding conducteur/passager.] [Figure A.2 : Messagerie temps réel.] [Figure A.3 : Tableau de bord conducteur / analytics.]

Annexe B : Extraits de code source commentés

Génération sécurisée de l'OTP (controllers/authController.js) :

```
const crypto = require('crypto');
// OTP cryptographiquement sûr (crypto.randomInt, pas
Math.random)
function generateOtp() {
  return String(crypto.randomInt(0, 1_000_000)).padStart(6,
'0');
}
```

Prévention de l'énumération de comptes (mot de passe oublié) :

```
async function forgotPassword(req, res, next) {
  const email = normalizeEmail(req.body.email);
  const user = await User.findOne({ email });
  // Réponse identique que le compte existe ou non
  res.json({ message: 'Si un compte existe, un code a été
envoyé.' });
  if (!user) return;
  // ... génération + envoi du code uniquement si le compte
existe
}
```

Prompt de classification vision KYC (CIN) :

Tu analyses la photo d'une Carte d'Identité Nationale (CIN) marocaine.

Réponds UNIQUEMENT avec un JSON: {"valide": true|false, "raison": "..."}.

"valide" = true seulement si l'image montre clairement un document

marocain lisible (recto avec photo, nom, numéro).

Annexe C : Configuration du système (reproductibilité)

Prérequis :

- Node.js ≥ 20
- Docker (pour MongoDB en replica set), ou un cluster MongoDB Atlas
- (Optionnel IA locale) Ollama + modèles : ollama pull llama et ollama pull llama3

Installation & lancement :

```
# 1. Cloner le dépôt
git clone https://github.com/yassineH123/Projet-fin-d-annee.git
cd Projet-fin-d-annee

# 2. Installer les dépendances (monorepo)
npm install

# 3. Configurer les variables d'environnement
cp backend/.env.example backend/.env # JWT_SECRET, MONGO_URI,
STRIPE_*, GROQ_API_KEY, CLOUDINARY_*

# 4. Démarrer MongoDB (replica set) via Docker
docker run -d --name atlasway-mongo -p 27018:27018 mongo --port
27018 --replSet rs0
# puis initialiser : rs.initiate({_id:"rs0", members:[{_id:0,
host:"localhost:27018"}]})

# 5. Lancer le backend (port 4000) et le web (port 5173)
```

```
npm run dev:backend  
npm run dev:web
```